"Drishti – AI Based Survillance System"

A Major Project Report Submitted to Rajiv Gandhi Proudyogiki Vishwavidyalaya

Towards Partial Fulfillment for the Award of

Bachelor of Technology in Computer Science Engineering

Submitted by:

Riya Kaushal (0827CS221230)
Mandisha Mirza(0827CS233D14)
Priyal Waghmare(0827CS221200)
Reeti Shrimal (0827CS221223)
Priyanka Chirote(0827CS221201)

Guided by

Prof. Akshay Dubey

Computer Science and Engineering

Acropolis Institute of Technology & Research, Indore

(Jul - Dec 2025)

EXAMINER APPROVAL

The Major Project entitled ***"Drishti – AI Based Survillance System"*** submitted by **Riya Kaushal (0827CS221230), Mandisha Mirza (0827CS233D14), Priyal Waghmare (0827CS221200), Reeti Shrimal (0827CS221223), Priyanka Chirote(0827CS221201)** has been examined and is hereby approved towards partial fulfillment for the award of ***Bachelor of Technology degree in Computer Science Engineering*** discipline, for which it has been submitted. It understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) **(External Examiner)**

Date: **Date:**

RECOMMENDATION

This is to certify that the work embodied in this major project entitled **"*Drishti – AI Based Survillance System*"** submitted by **Riya Kaushal (0827CS221230), Mandisha Mirza (0827CS233D14), Priyal Waghmare (0827CS221200), Reeti Shrimal (0827CS221223), Priyanka Chirote (0827CS221201)** is a satisfactory account of the bonafide work done under the supervision of ***Prof. Akshay Dubey***, is recommended towards partial fulfillment for the award of the Bachelor of Technology (Computer Science Engineering) degree by Rajiv Gandhi Proudyogiki Vishwavidhyalaya, Bhopal.

(Project Guide)

(Project Coordinator)

(Dean Academics)

STUDENTS UNDERTAKING

This is to certify that the major project entitled ***"Drishti – AI Based Survillance System"*** has developed by us under the supervision of ***Prof. Akshay Dubey***. The whole responsibility of the work done in this project is ours. The sole intension of this work is only for practical learning and research.

We further declare that to the best of our knowledge; this report does not contain any part of any work which has been submitted for the award of any degree either in this University or in any other University / Deemed University without proper citation and if the same work found then we are liable for explanation to this.

Riya Kaushal (0827CS221230)

Mandisha Mirza (0827CS233D14)

Priyal Waghmare(0827CS221200)

Reeti Shrimal (0827CS221223)

Priyanka Chirote(0827CS221201)

Acknowledgement

We thank the almighty Lord for giving me the strength and courage to sail out through the tough and reach on shore safely.

There are number of people without whom this project would not have been feasible. Their high academic standards and personal integrity provided me with continuous guidance and support.

We owe a debt of sincere gratitude, deep sense of reverence and respect to our guide and mentor **Prof Aksahy Dubey,** Professor, AITR, Indore for his motivation, sagacious guidance, constant encouragement, vigilant supervision, and valuable critical appreciation throughout this project work, which helped us to successfully complete the project on time.

We express profound gratitude and heartfelt thanks to **Dr Kamal Kumar Sethi**, Professor & Head CSE, AITR Indore for his support, suggestion, and inspiration for carrying out this project. I am very much thankful to other faculty and staff members of the department for providing me all support, help and advice during the project. We would be failing in our duty if do not acknowledge the support and guidance received from **Dr S C Sharma**, Director, AITR, Indore whenever needed. We take opportunity to convey my regards to the management of Acropolis Institute, Indore for extending academic and administrative support and providing me all necessary facilities for project to achieve our objectives.

We are grateful to **our parent** and **family members** who have always loved and supported us unconditionally. To all of them, we want to say “Thank you”, for being the best family that one could ever have and without whom none of this would have been possible.

Executive Summary

"Drishti – AI Based Survillance System"

This project is submitted to **Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal**, in partial fulfillment of the Bachelor of Engineering in Computer Science , under the guidance of **Prof. Akshay Dubey**.

Drishti is an AI-powered smart surveillance system designed for **real-time face recognition, headcounting, and intelligent monitoring**. The system uses **Deep Learning** concepts with **TensorFlow**, **YOLOv8**, and **DeepFace** for fast and accurate detection through live camera feeds. A pre-trained model based on the **COCO dataset** enables robust identification of students, supporting automated attendance and enhanced campus security. A built-in dashboard displays live analytics for seamless monitoring.

Key words: Face Recognition, AI Surveillance, Real-Time Detection, Intelligent Monitoring, YOLOv8, DeepFace, Dashboard Analytics, Privacy-Preserving AI, Automated Attendance, Smart Security.

"Where the vision is one year,

cultivate flowers;

Where the vision is ten years,

cultivate trees;

Where the vision is eternity,

cultivate people."

- Oriental Sayin

List of Figures

Figure 1.1: Architecture of the DRISHTI Smart Surveillance System

Figure 2.1: Overview of Existing Surveillance Systems

Figure 2.2: YOLOv8 Object Detection Workflow

Figure 2.3: Facial Recognition Pipeline using DeepFace

Figure 3.1: Block Diagram of the Proposed DRISHTI System

Figure 3.2: System Data Flow Diagram (DFD – Level 0)

Figure 3.3: System Data Flow Diagram (DFD – Level 1)

Figure 3.4: Use-Case Diagram for User and Admin Roles

Figure 4.1: Real-Time Video Input Processing using OpenCV

Figure 4.2: Face Detection and Recognition Module

Figure 4.3: Headcount Detection Using YOLOv8

Figure 4.4: Bounding Box Output for Detected Faces

Figure 4.5: Workflow of Data Storage in MongoDB/Database

Figure 5.1: Dashboard Interface – Live Camera Stream

Figure 5.2: Dashboard Interface – Recognized Persons Display

Figure 5.3: Alerts and Unknown Person Notification Interface

Figure 5.4: Historical Logs and Analytics Module

Figure 6.1: Accuracy Comparison of Detection and Recognition Models

Figure 6.2: Confusion Matrix for Face Recognition

Figure 7 : Screenshots

Figure 7.1 : Output Figures

Figure 8 : Gantt Chart

List of Tables

Table 4.1 Comparison of Face Recognition Techniques 43

Table 4.2 Comparison of Headcounting Algorithms 44

Table 4.3 Dashboard Performance Compariso 45

Table 4.4 Speed Comparison 46

Table 4.5 Test Case 1 – Face Recognition Accuracy Test 47

Table 4.6 Test Case 2 – Headcount Accuracy Verification 48

Table 4.7 Test Case 3 – Dashboard System Testing 49

Table 5.1 Bibliography 52

List of Abbreviations

***Abbr* AI – Artificial Intelligence**

***Abbr* ML – Machine Learning**

***Abbr* CNN – Convolutional Neural Network**

***Abbr* UI – User Interface**

***Abbr* UX – User Experience**

***Abbr* LSTM – Long Short-Term Memory**

***Abbr* LBPH – Local Binary Patterns Histogram**

***Abbr* YOLO – You Only Look Once**

***Abbr* API – Application Programming Interface**

***Abbr* DB – Database**

***Abbr* SSD – Single Shot Detector**

***Abbr* FPS – Frames Per Second**

***Abbr* IDE – Integrated Development Environment**

***Abbr* NLP – Natural Language Processing**

***Abbr* RNN – Recurrent Neural Network**

Table of Contents

CHAPTER 1 – INTRODUCTION

1.1 Overview…….....13
1.2 Background and Motivation……...14
1.3 Problem Statement and Objectives…….14
1.4 Scope of the Project……....15
1.5 Features of the Project……16
1.6 Team Organization…….17
1.7 Report Structure……....…..18

CHAPTER 2 – REVIEW OF LITERATURE

2.1 Preliminary Investigation…….……...…....19
2.1.1 Current System……...….....19
2.2 Limitations of Current System…….....20
2.3 Requirement Identification and Analysis……..…..20
2.3.1 Literature Review Summary……...….....20
2.3.1 Conclusion……...22

CHAPTER 3 – PROPOSED SYSTEM

3.1 The Proposal ..... …….23
3.2 Benefits of the Proposed System..... 23
3.3 System Overview 25
3.4 System Workflow..... 26
3.5 Feasibility Study 27
3.6 System Architecture…….29
3.7 Development Diagram……...……..31

3.8 Deployment Requirements……..….36

CHAPTER 4 – IMPLEMENTATION

4.1 Techniques Used……..….38
4.1.1 Face Detection Technique…….....38
4.1.2 Facial Recognition Technique…….....39
4.1.3 Person Detection & Headcount (YOLO).....….....39
4.1.4 Real-time Video Streaming……...41
4.1.5 Backend API Technique……....41
4.1.6 Database Technique……..41
4.1.7 Dashboard Visualization……..... 42

4.2 Tools Used 42
4.3 Performance Comparison of Implemented Technique..... 46
4.4 Graphical Comparison 48
4.5 Screenshots……..……50
4.6 Testing……..52

CHAPTER 5 – CONCLUSION & FUTURE SCOPE

5.1 Conclusion……..57
5.2 Limitations……..57
5.3 Suggestions & Recommendations……..58

BIBLIOGRAPHY & APPENDICES

Bibliography…….61
Project Plan & Gantt Chart……...62
Guide Interaction Sheet……62
Appendix / Source Code……64

Chapter 1.

Introduction

In today's world, ensuring safety and security in public and private spaces has become a critical concern. Traditional surveillance systems rely heavily on manual monitoring, which is time-consuming, error-prone, and limited by human attention. With the rapid advancement of Artificial Intelligence (AI) and Computer Vision, modern surveillance can be made more intelligent, adaptive, and efficient.

The proposed system, **Drishti**, is an AI-based smart surveillance solution designed to automatically detect, recognize, and track individuals in real-time. It integrates **YOLOv8 (You Only Look Once)** for object and human detection, along with **DeepFace**, an advanced face recognition framework, to identify individuals accurately. By combining these technologies, Drishti provides a robust and automated approach to enhance security and monitoring in environments such as educational institutions, offices, residential complexes, and public areas.

1.1 Overview

Drishti offers a seamless, intelligent, and automated surveillance mechanism that continuously analyzes live video feeds from cameras. The system is capable of:

- Detecting human presence or unauthorized entries.
- Recognizing faces of known and unknown individuals.
- Storing captured data and recognition logs in a secure database for future reference.
- Generating alerts or notifications when suspicious activity or unknown faces are detecte

The complete workflow involves capturing live video input, processing it through the **YOLOv8** model for object detection, and using **DeepFace** for facial recognition. The results are displayed on a web-based dashboard, enabling security personnel to monitor real-time activities efficiently. The system's modular design allows easy integration with existing CCTV setups, making it both cost-effective and scalable.

1.2 Background and Motivation

Conventional CCTV systems depend on continuous human supervision to identify incidents, which can result in delayed responses or missed events. In an era where automation and real-time analytics dominate every domain, there is a growing demand for intelligent surveillance solutions.

The motivation behind **Drishti** stems from the need to reduce manual monitoring efforts and enhance accuracy in identifying individuals. Integrating AI with surveillance not only increases security but also supports data-driven decision-making. By using **Deep Learning models**, Drishti ensures quick, reliable, and precise recognition even in challenging conditions such as low light or varying facial expressions.

Moreover, the rise in unauthorized access, campus incidents, and public security concerns has encouraged the development of smart monitoring systems that provide real-time alerts and maintain digital evidence for investigations.

1.3 Problem Statement and Objectives

Existing surveillance systems rely heavily on manual monitoring, leading to inefficiencies, human errors, and delayed responses. They lack intelligent recognition capabilities and are unable to differentiate between authorized and unauthorized individuals. There is a need for an AI-based automated surveillance system that can perform real-time detection, recognition, and analysis to enhance security and operational efficiency.

Objectives:

The main objectives of **Drishti** are:

- To develop an AI-powered surveillance system capable of real-time human detection and facial recognition.
- To integrate **YOLOv8** and **DeepFace** for accurate recognition under different lighting and environmental conditions.
- To automate security monitoring by eliminating the need for continuous manual observation.
- To design a user-friendly web interface for live monitoring and alert management.
- To ensure efficient data logging and retrieval for investigation and reporting purposes.

1.4 Scope of the Project

The scope of **Drishti** extends across multiple domains requiring smart surveillance and identity verification. Some key application areas include:

- **Educational Institutions:** Automated monitoring of attendance and campus security.
- **Corporate Offices:** Access control and employee movement tracking.
- **Residential Areas:** Detecting intrusions or unauthorized entries.
- **Airports and Railways:** Crowd monitoring and detection of suspicious activity.
- **Hospitals and Banks:** Real-time facial recognition for security verification.
- **Public Spaces:** Enhanced situational awareness in parks, malls, and streets.

The system can be deployed on both small-scale local networks and large-scale cloud infrastructures, ensuring scalability and reliability.

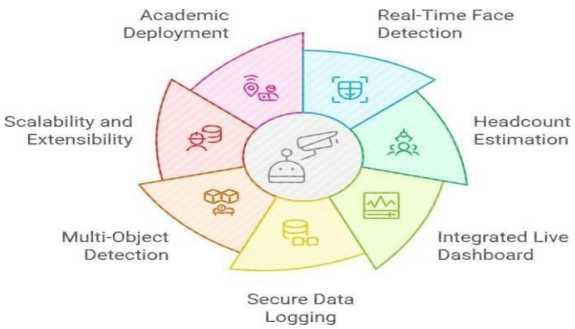


Figure 1-1 : Scope of the project

1.5 Feature of The Project

Real-Time Person Detection

- Detects individuals in live video streams using YOLOv8.
- Provides instant and continuous headcount to monitor crowd density.

Facial Recognition

- Identifies known individuals using the DeepFace library.
- Automatically labels unrecognized individuals as **"Unknown"** to maintain privacy.

Interactive Dashboard

- Offers a web-based interface displaying the live camera feed.
- Shows real-time headcount, alerts, and recognized individuals.
- Presents historical logs, analytics, and statistics for review.

Privacy-Aware Monitoring

- Ensures that data of unknown individuals is not stored.
- Maintains compliance with privacy and security guidelines.

Database Management

- Stores details of recognized individuals along with timestamps.
- Saves headcount records and detection logs securely for future use.

User-Friendly Interface

- Clean, intuitive, and responsive UI for seamless monitoring.
- Requires minimal user interaction for effective system operation.

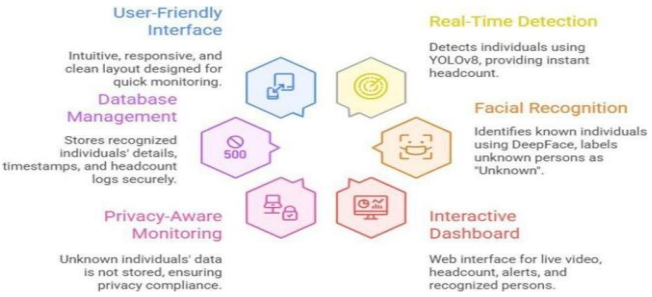


Figure 1-2: Feature of the project

1.6 Team Organization

Priyal Waghmare :

The Team Leader and Backend & Database Developer, was responsible for leading the project, managing coordination among team members, and overseeing the entire development process. She handled the backend logic, API integration, and database management using MongoDB, ensuring secure and efficient data storage, retrieval, and system communication.Yash Bhawsar :

Reeti Shrimal:

Serving as the **AI Model Developer**, focused on the design, training, and optimization of artificial intelligence models. She implemented **YOLOv8** for object detection and **DeepFace** for facial recognition, ensuring accurate and real-time detection performance within the surveillance system.

Riya Kaushal:

Working as a **Full Stack Developer (UI/UX Focus)**, contributed to both **frontend and backend development**. She designed the **user interface** and improved the **user experience** through interactive layouts and data visualization. Additionally, she played a vital role in preparing the **technical documentation** of the project.

Mandisha Mirza:

Working as a **Full Stack Developer**, worked on integrating the AI modules with the web application and ensuring smooth **system deployment**. She collaborated closely on the frontend and backend functionalities and contributed to **technical documentation** along with Riya and Priyanka.

Priyanka Chirote:

The **Documentation Specialist**, was responsible for preparing, compiling, and formatting all project-related documents. She worked alongside **Riya Kaushal** and **Mandisha Mirza** to ensure that the technical documentation was comprehensive, accurate, and professionally presented.

1.7 Report Structure

The project report for **Drishti: AI-Based Smart Surveillance System** is systematically organized into five chapters, each detailing different aspects of the system's design, development, and evaluation.

Chapter 1: Introduction – This chapter provides an overview of the project, including the background, motivation, objectives, and scope. It also introduces the problem statement that inspired the development of *Drishti* and explains the need for an automated, AI-based surveillance system. Furthermore, it presents the team organization and outlines the structure of the entire report.

Chapter 2: Review of Literature – This chapter discusses the research and technological advancements in the field of computer vision, object detection, and facial recognition. It highlights the existing systems, their limitations, and how *Drishti* overcomes those challenges using modern AI techniques such as YOLOv8 and DeepFace. The chapter concludes with the identification of specific requirements that guided the development of the proposed system.

Chapter 3: Proposed System – This chapter explains the conceptual design and architecture of *Drishti*. It presents the system workflow, block diagrams, and detailed module descriptions, including object detection, facial recognition, and database management. The chapter also discusses the benefits, feasibility studies, and deployment requirements of the system, illustrating how the proposed design ensures real-time performance and accuracy.

Chapter 4: Implementation – This chapter elaborates on the technologies, frameworks, and programming tools used in the implementation of *Drishti*. It includes details of YOLOv8 and DeepFace integration, MongoDB database setup, and full-stack development using web technologies. The user interfaces are presented along with their functionalities, followed by system testing, result analysis, and performance evaluation based on accuracy, efficiency, and scalability.

Chapter 5: Conclusion and Future Scope – The final chapter summarizes the project outcomes with respect to its initial objectives. It highlights the key findings, results achieved, and limitations encountered during development. Additionally, it provides insights into the potential future enhancements of *Drishti*, such as multi-camera integration, cloud-based analytics, and extended real-time monitoring capabilities

Chapter 2.

Review of Literature

The development of intelligent surveillance systems has become a major focus in the field of computer vision and artificial intelligence. Over the past decade, numerous studies and research initiatives have been undertaken to improve the accuracy, efficiency, and automation of object detection and facial recognition technologies. The purpose of this chapter is to review existing literature, identify the strengths and weaknesses of previous approaches, and highlight how *Drishti: AI-Based Smart Surveillance System* builds upon these foundations to deliver a more reliable and real-time monitoring solution.

2.1 Preliminary Investigation

2.1.1 Current System

The current surveillance systems in many institutions and public areas primarily depend on **manual monitoring** and **traditional CCTV cameras**. These systems merely record footage without offering **real-time intelligence** such as automatic object recognition, facial identification, or activity alerts. Consequently, incidents are often reviewed **after** they occur, leading to delays in decision-making and compromised security response times.

Most existing systems lack integration with **AI-based automation** and rely on human operators for continuous observation. This dependence increases the likelihood of oversight and fatigue-related errors. Moreover, conventional CCTV setups do not maintain intelligent databases for logging events, faces, or objects detected in real-time.

To overcome these limitations, the proposed **Drishti System** integrates **YOLOv8- based object detection**, **DeepFace-based recognition**, and **real-time database logging** to automate surveillance and improve operational efficiency.

2.2 Limitations of Current System

The limitations observed in existing surveillance systems are summarized below:

- **Manual Dependence:** Current systems require human operators to monitor live footage continuously, which is inefficient and error-prone.
- **Lack of Real-Time Analytics:** Traditional CCTV systems record data without real-time recognition or alert mechanisms.
- **No Automated Event Detection:** They cannot automatically detect or categorize suspicious activity, unknown faces, or intrusions.
- **Poor Scalability:** Manual or basic systems cannot easily scale across multiple cameras or locations.
- **Data Management Issues:** Recorded video files are often stored without metadata or event logs, making post-event retrieval cumbersome.

The *Drishti* system addresses these challenges by incorporating intelligent recognition models, automated tracking, and a central database for event management and visualization.

2.3 Requirement Identification and Analysis for Project

Research in **object detection**, **facial recognition**, and **computer vision-based monitoring** has evolved rapidly with advancements in deep learning. Traditional algorithms relied heavily on handcrafted features and static backgrounds, but modern systems use neural networks capable of learning complex patterns.

The following literature review highlights key developments that influenced the design of *Drishti: AI-Based Smart Surveillance System.*

System	Technology Used	Functionality	Limitations
Manual Surveillance	Human Observation, Registers	Security personnel manually monitor and record people or vehicles entering premises.	Time-consuming, prone to human error, not scalable, cannot operate 24×7.
Basic CCTV Monitoring	Camera + DVR/NVR	Captures and stores footage for later viewing; no real-time analysis.	Requires manual review; no automated detection or alert generation.

Motion Detection Systems	Infrared Sensors, Basic Vision Algorithms	Detects movement in restricted areas and triggers alarms.	Cannot identify object type (person, vehicle, or animal); high false alarms.
Haar Cascade Classifier (Old Vision Models)	OpenCV, Haar Features	Detects human faces or objects using pattern-based detection.	Poor accuracy in varying light conditions; limited real-time performance.
Traditional ML-Based Systems	SVM, HOG, or CNN (basic)	Detects pre-trained objects in controlled environments.	Struggles with real-world noise, occlusions, and large datasets.

2.3.1 Literature Review Summary

1. **Object Detection Using Deep Learning:** Redmon et al. introduced **YOLO (You Only Look Once)** as a unified, real-time object detection model capable of predicting bounding boxes and class probabilities in a single evaluation [1]. The approach revolutionized surveillance analytics by significantly improving speed and reducing false detections. Modern versions such as **YOLOv8** achieve higher accuracy and can process multiple classes simultaneously.
2. **Facial Recognition Systems:** Taigman et al. presented DeepFace, a deep learning architecture capable of face verification with near-human accuracy [2]. It uses convolutional neural networks (CNNs) to map facial features and match them even under variations in pose, lighting, and expression. This concept serves as the foundation for the facial recognition module in *Drishti.*
3. **Object Tracking and Real-Time Detection**: Wu et al. developed a hybrid tracking algorithm combining motion differencing and correlation matching, ensuring consistent tracking of moving entities in dynamic scenes [3]. This inspired the movement-tracking component integrated within *Drishti* for continuous surveillance.
4. **Intelligent Video Surveillance (IVS):** Recent studies have emphasized IVS frameworks that integrate AI with video analytics for crowd monitoring, behavioral analysis, and intrusion detection [4].
5. **Intelligent Video Surveillance (IVS):** Recent studies have emphasized IVS frameworks that integrate AI with video analytics for crowd monitoring, behavioral analysis, and intrusion detection [4]. These systems demonstrate how real-time alerts and automatic event classification improve campus and corporate security systems.

6. **Intelligent Video Surveillance (IVS):** Recent studies have emphasized IVS frameworks that integrate AI with video analytics for crowd monitoring, behavioral analysis, and intrusion detection [4]. These systems demonstrate how real-time alerts and automatic event classification improve campus and corporate security systems.

7. **Intelligent Video Surveillance (IVS):** Recent studies have emphasized IVS frameworks that integrate AI with video analytics for crowd monitoring, behavioral analysis, and intrusion detection [4]. These systems demonstrate how real-time alerts and automatic event classification improve campus and corporate security systems.
8. **Deep Neural Network-Based Human Detection:** Dalal and Triggs' Histogram of Oriented Gradients (HOG) [5] provided a foundation for human detection, later improved by CNN-based models. Combined with YOLO architectures, they achieve superior detection of humans, vehicles, and animals in variable lighting and environments.
9. **Real-Time Database Integration for Surveillance:** Research by Sharma et al. [6] proposed the use of cloud databases such as MongoDB and Firebase for storing metadata, facial identities, and event logs. This approach ensures that surveillance systems are not only intelligent but also data-driven and scalable — a concept applied directly in *Drishti.*
10. **Hybrid Security and IoT-Based Monitoring:** Kumar and Singh [7] explored IoT-enabled camera systems that transmit real-time object and face data to cloud servers for predictive analytics. Their study highlighted the importance of system interoperability and end-to-end integration, both of which are implemented in *Drishti.*

2.3.1 Conclusion

This chapter presented an extensive review of the existing systems, technologies, and research work carried out in the domains of object detection, facial recognition, and intelligent surveillance. The literature clearly indicates a gradual evolution from traditional manual monitoring and basic CCTV systems to advanced deep-learning-based solutions such as YOLO, Faster R-CNN, and Deep Face. While the earlier systems were limited by human dependency, low accuracy, and lack of automation, the newer approaches demonstrate significant improvements in precision, speed, and scalability. However, most of these

systems still lack real-time integration of multiple functionalities, face recognition, and automated data logging—within a unified framework.

The findings from this review highlight the need for a comprehensive, end-to-end solution capable of performing intelligent surveillance with minimal human intervention. This gap in current research and implementation motivates the development of the proposed system, **Drishti**, which integrates **YOLOv8**, **DeepFace**, and **MongoDB** to deliver a robust, real-time, AI-driven surveillance and monitoring platform

Chapter 3.

Proposed System

3.1 The Proposal

The proposed system, **Drishti**, is an AI-powered smart surveillance solution designed to overcome the limitations of traditional CCTV systems. Unlike conventional setups that require manual monitoring, Drishti integrates **real-time person detection, automatic headcount generation, and AI-based facial recognition** to ensure intelligent, automated, and privacy-compliant security.

The system leverages **YOLOv8** for high-speed person detection and **DeepFace** for face identification. All processed information—including headcount, recognized identities, and alerts—is displayed on a **web-based interactive dashboard**, enabling seamless monitoring from any connected device.

3.2 Benefits of the Proposed System

The proposed system **Drishti** offers several significant advantages over traditional surveillance mechanisms. By combining AI-driven person detection, automated headcounting, and facial recognition with a real-time dashboard, the system ensures higher security, efficiency, and reliability. The key benefits are as follows:

1. Enhanced Security & Monitoring

- Provides continuous and automated surveillance without requiring constant human supervision.
- Instantly identifies known individuals and flags unknown persons, enabling quick action in sensitive areas.

2. Real-Time Intelligence

- YOLOv8 ensures rapid detection and tracking of individuals in the scene.
- Automatic headcount provides immediate insight into crowd density, helping prevent overcrowding or security breaches.

3. Reduced Manual Effort

- Eliminates the need for operators to manually monitor CCTV footage.
- Automated recognition and real-time alerts reduce workload on security personnel.

4. High Accuracy & Speed

- Deep learning models significantly outperform traditional detection and face recognition methods.
- Ensures high accuracy even in dynamic environments.

5. Privacy-Preserving Design

- Unknown individuals are labeled as **"Unknown"** and NOT stored in the database.
- System maintains data integrity and complies with privacy considerations.

6. Centralized Web Dashboard

- Offers a clean, interactive interface displaying:
  - Live camera feed
  - Real-time headcount
  - Recognition results
  - Alerts/status indicators
- Enables efficient and informed decision-making.

7. Scalable & Modular Architecture

- Designed for easy integration with future enhancements:
  - Multi-camera support
  - Cloud-based analytics
  - Mobile notifications
- Components can be upgraded without altering the entire system.

8. Cost-Effective Implementation

- Works on standard hardware like webcams and mid-range computers.
- Reduces the need for expensive proprietary surveillance equipment.

9. Improved Safety in Public/Private Spaces

- Useful for institutions such as classrooms, offices, labs, hostels, small

events, and restricted areas.

- Helps ensure safety by identifying unauthorized access or unusual crowding

10. Historical Data for Analysis

- Logs recognized individuals and timestamps for audits and record keeping.
- Valuable for security investigations or attendance-style tracking.

3.3 Block Diagram

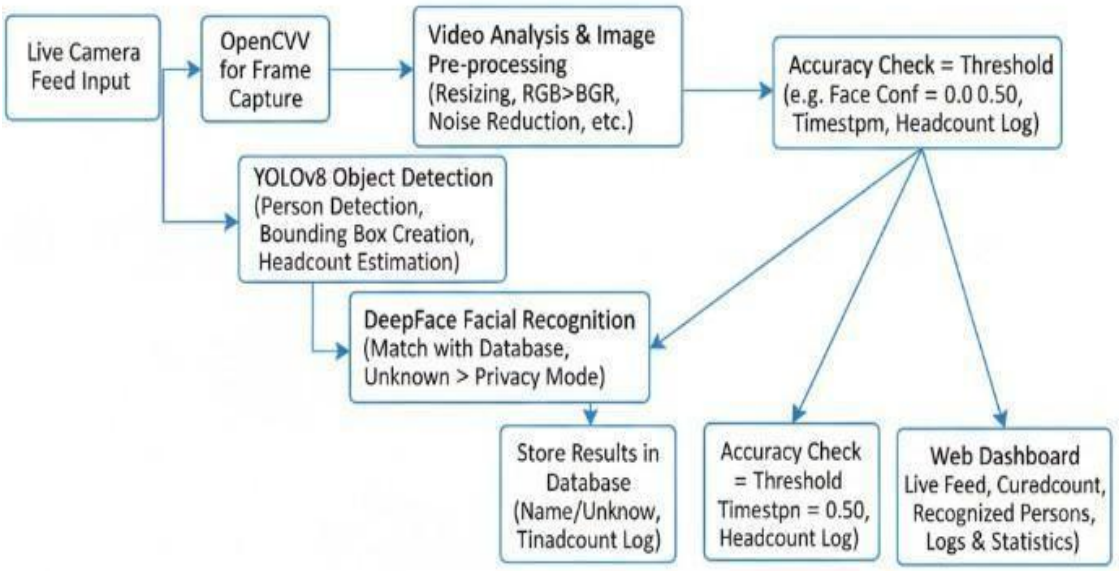


Figure 3-1 : Block Diagram

3.2 System Overview

The proposed system receives live video input from a webcam or IP camera, processes each frame using AI/ML models, and generates real-time output for surveillance operators.

Key functions include:

- Detecting all individuals present in the camera frame.
- Counting total people in real time.
- Recognizing known individuals using a pre-trained face dataset.
- Labeling unrecognized individuals as **"Unknown"** to maintain privacy.
- Displaying the processed information on a real-time dashboard.
- Storing recognized identities securely for reporting and audits.

The system follows a **modular, scalable architecture**, allowing easy

integration of future features like multi-camera support, cloud-based storage, and mobile alerts.

3.3 System Workflow

The entire flow of the proposed system can be summarized in the following steps:

Step 1: Video Capture

- A live video stream is captured using a webcam or IP camera.
- Frames are continuously fed into the backend processing module.

Step 2: Real-Time Person Detection (YOLOv8)

- YOLOv8 analyzes each frame to detect humans with high accuracy.
- Bounding boxes are generated for each person.
- Real-time headcount is computed.

Step 3: Facial Recognition (DeepFace)

- Detected face regions are extracted from the frame.
- DeepFace compares the embeddings with the stored dataset.
- If a match is found → recognized person name/details are shown.
- If not matched → labeled as **"Unknown"** (no data stored).

Step 4: Data Storage

- Only recognized individuals' details and timestamps are saved.
- Database maintains logs for future reports and analysis.
- Unknown individuals are never stored, ensuring privacy.

Step 5: Dashboard Visualization

- Web dashboard displays:
  - Live video feed
  - Headcount
  - Recognized individuals

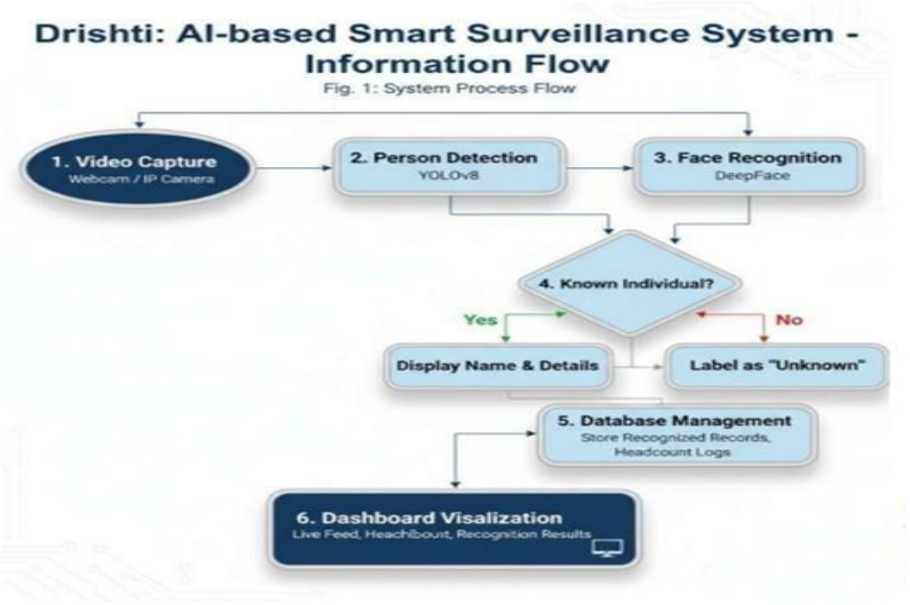


Figure 3-2 : Interaction Overview

3.4 Feasibility Study

A feasibility study is conducted to evaluate whether the proposed system **Drishti: AI-Based Smart Surveillance System** can be successfully developed, implemented, and maintained within the available constraints. The study analyzes the system from **technical**, **operational**, **economic**, **legal**, and **time** perspectives to ensure its practicality and effectiveness.

3.4.1 Technical

Technical feasibility examines whether the required technology, tools, and expertise are available to implement the system.

Findings:

- The system utilizes **YOLOv8**, **DeepFace**, **OpenCV**, and **Python**, which are open-source and well-documented.
- Modern laptops with CPU/GPU support are sufficient for real-time detection and recognition.
- Required technologies such as Flask/Django, HTML/CSS/JS, and MongoDB/MySQL are widely accessible.

- The team possesses the necessary technical skills in machine learning, web development, and database management.

Conclusion: The system is technically feasible, as all required components and competencies are available and easily implementable.

3.4.2 Economical

Economic feasibility determines whether the system is cost-effective and financially justified.

Findings:

- The system uses mostly **open-source frameworks**, which reduces development cost.
- No expensive hardware is required; a standard webcam and a mid-range system are sufficient.
- Long-term benefits include reduced human monitoring cost and improved security.
- Maintenance costs are minimal and limited to updates and occasional hardware upgrades.

Conclusion: The system is economically feasible, offering high value at a low implementation and maintenance cost.

3.4.3 Operational

Operational feasibility evaluates whether the system will function effectively in the intended environment and whether users can easily adopt it.

Findings:

- The dashboard interface is user-friendly and requires minimal training for security personnel.
- Automated processes like headcount and recognition reduce manual workload.
- The system enhances existing security operations without disrupting daily activities.
- Real-time updates and alerts improve monitoring efficiency.

Conclusion: The system is operationally feasible, as it aligns with user needs and significantly enhances surveillance operations.

3.4.5 Time Feasibility

Time feasibility assesses whether the project can be completed within the allotted academic/project timeline.

Findings:

- The project plan and Gantt chart outline clear milestones.
- AI model setup, backend integration, and dashboard development can be completed within the given semester.
- Team members have distributed roles, reducing time pressure.
- Most tools and datasets are readily available, minimizing delays.

Conclusion: The project is time feasible and can be completed within the specified duration.

3.5 System Architecture

The system architecture of **Drishti: AI-Based Smart Surveillance System** follows a modular, layered, and scalable design that ensures seamless real-time video processing, AI-based person detection, facial recognition, and dashboard visualization. Each module communicates efficiently to deliver accurate results while maintaining privacy and security.

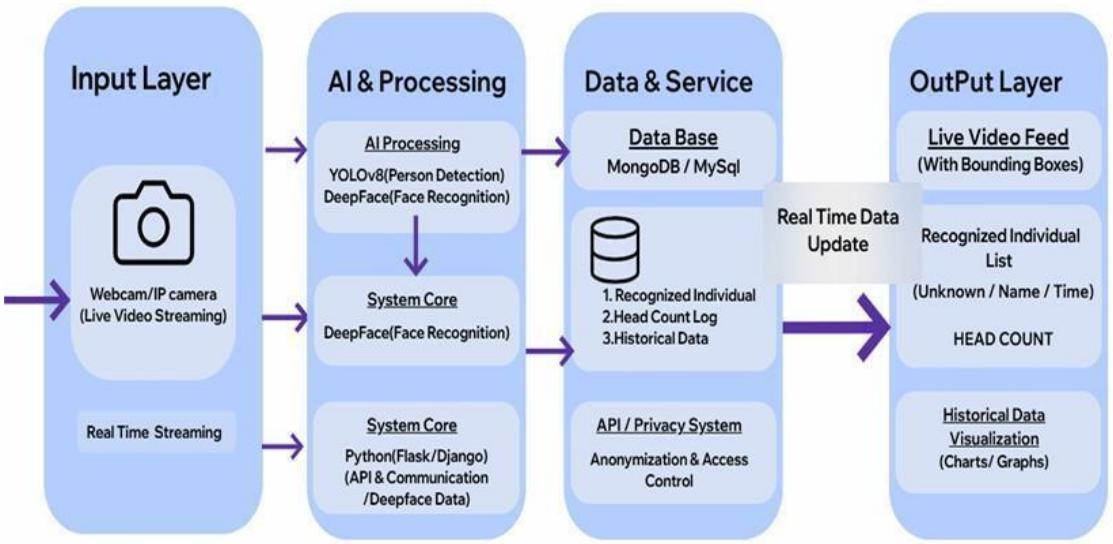


Figure 3-3 : System Architecture

1. Input Layer: Captures live video feed from a webcam or IP camera via the web application. Supports real-time monitoring of multiple individuals for crowd analysis and facial recognition.

2. Front-End (Client-Side): Built using HTML5, CSS (Tailwind), and JavaScript/React.js for a responsive, interactive UI. Dashboard shows live video feed, headcount, and recognized individuals. User-friendly layout with clear visual alerts and minimal text for quick monitoring.

3. Back-End (Server-Side): Developed using Python (Flask/Django/FastAPI) to handle video processing, AI computations, and API requests. Integrates YOLOv8 for person detection,and DeepFace for facial recognition. Manages database operations and real-time communication with frontend via APIs or WebSockets.

4. AI Processing Module: Person Detection (YOLOv8): Detects individuals in each frame for headcount and spatial analysis. Facial Recognition (DeepFace): Matches detected faces with a pre-built dataset and displays recognized individuals.

5. Database & Storage: Stores records of recognized individuals, timestamps, headcount logs, information. Maintains historical data for analysis and reporting. Ensures secure storage with encryption and access controls.

6. Security & Privacy Module: Labels unknown individuals as "Unknown" without storing their data. Secures communication between frontend and backend using HTTPS/SSL. Optional user authentication for restricted access to dashboard and database.

3.5.1 Data Flow Diagrams

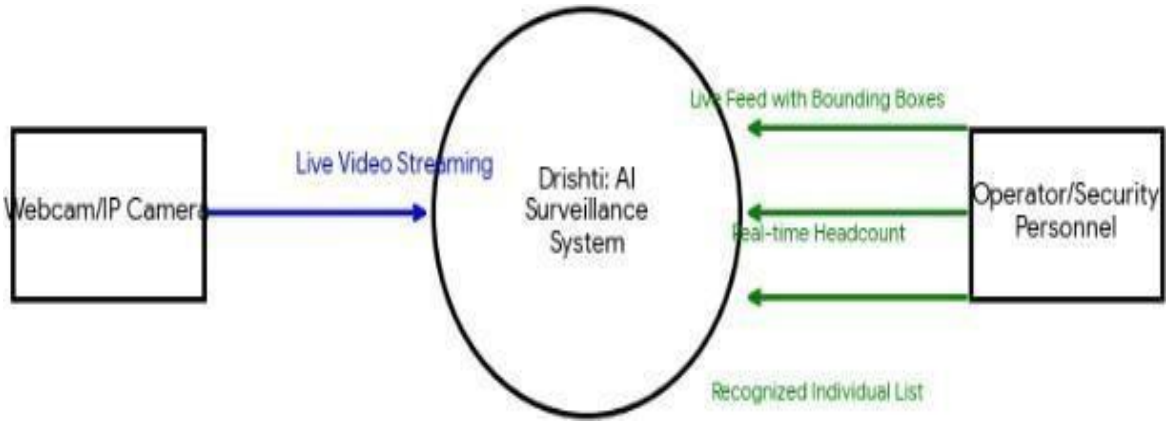


Figure 3-4 Data Flow Diagram Level

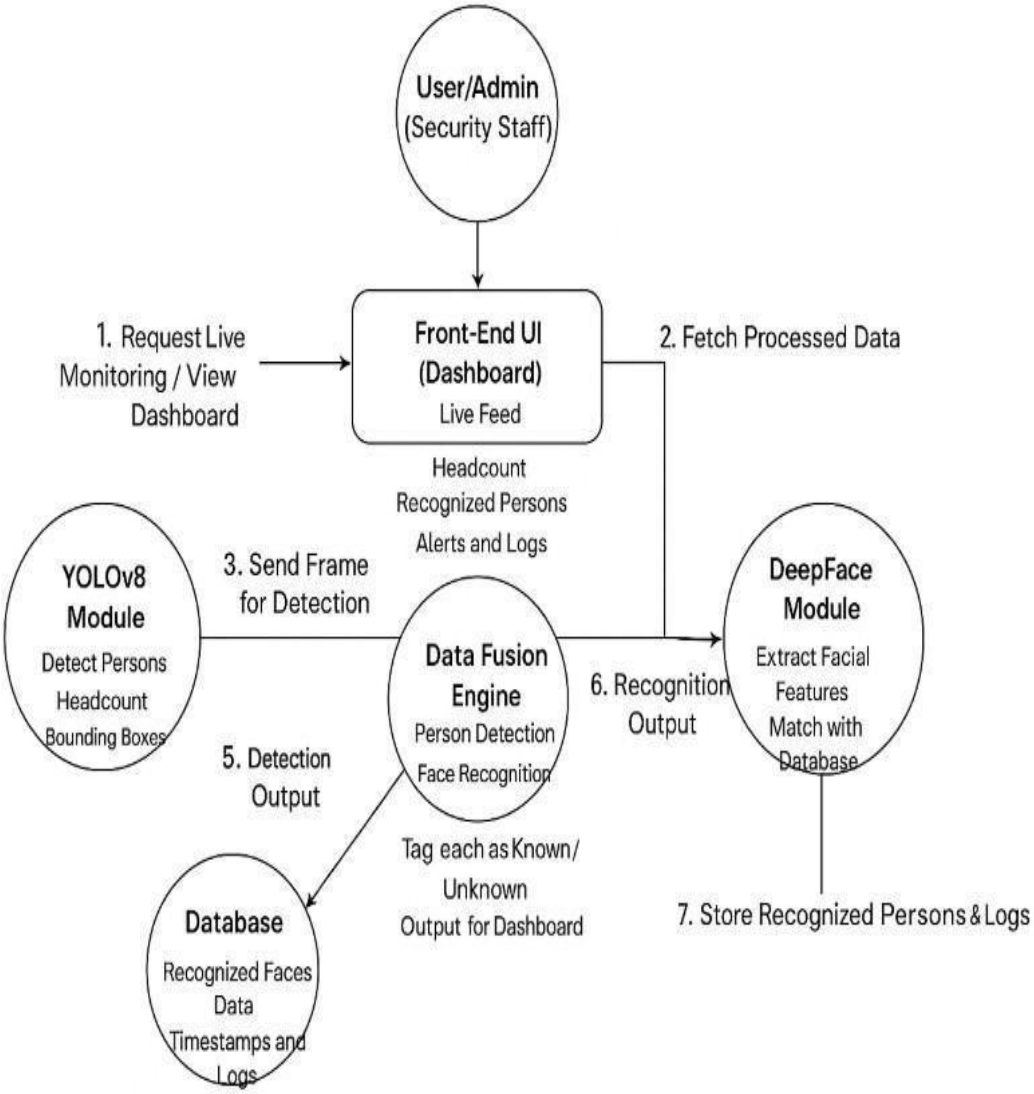


Figure 3-5 Data Flow Diagram Level 1

3.6 Development Diagrams (UML Diagrams)

3.6.1 Use Case Diagram:

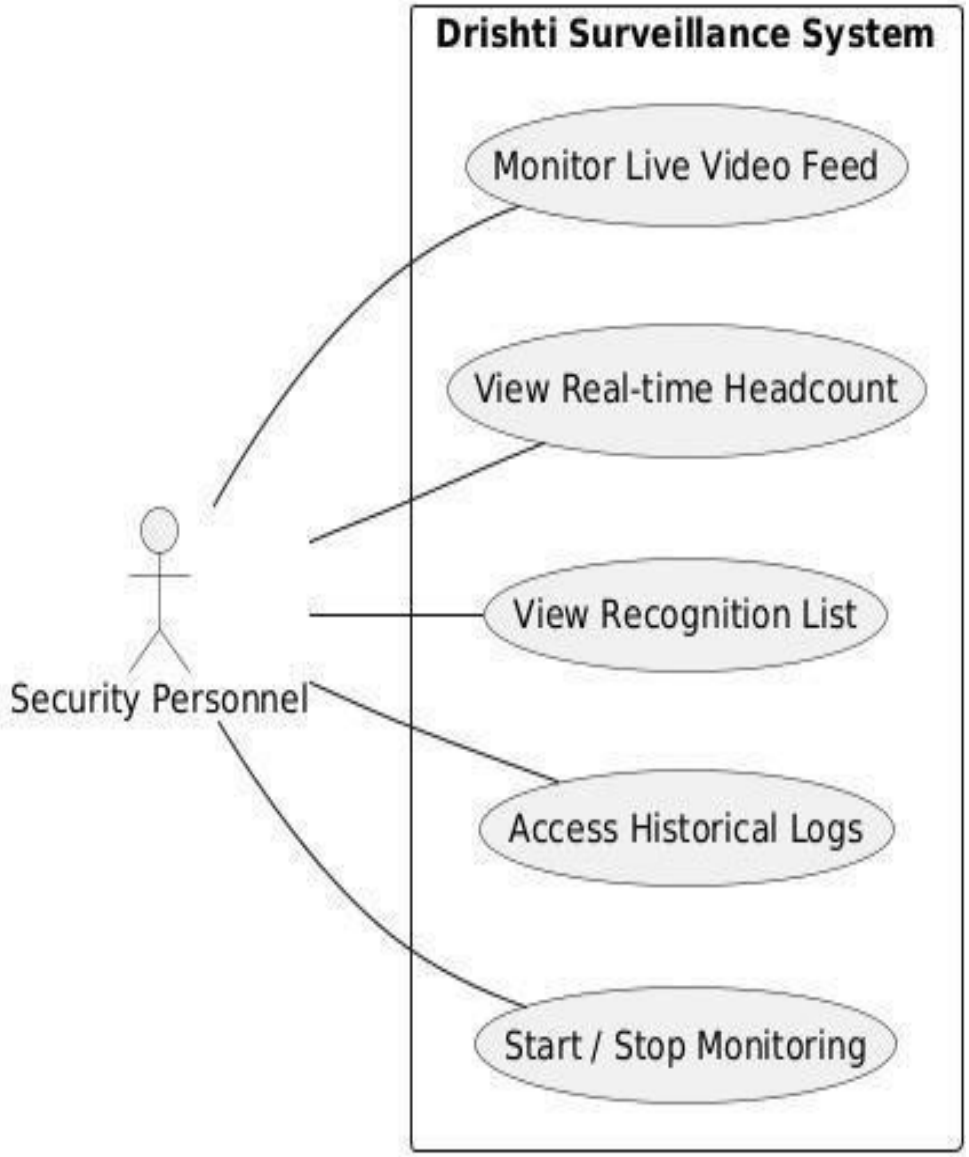


Figure 3-6 : Use Case Diagram

3.6.2 ER Diagram(Entity Relationship Diagram):

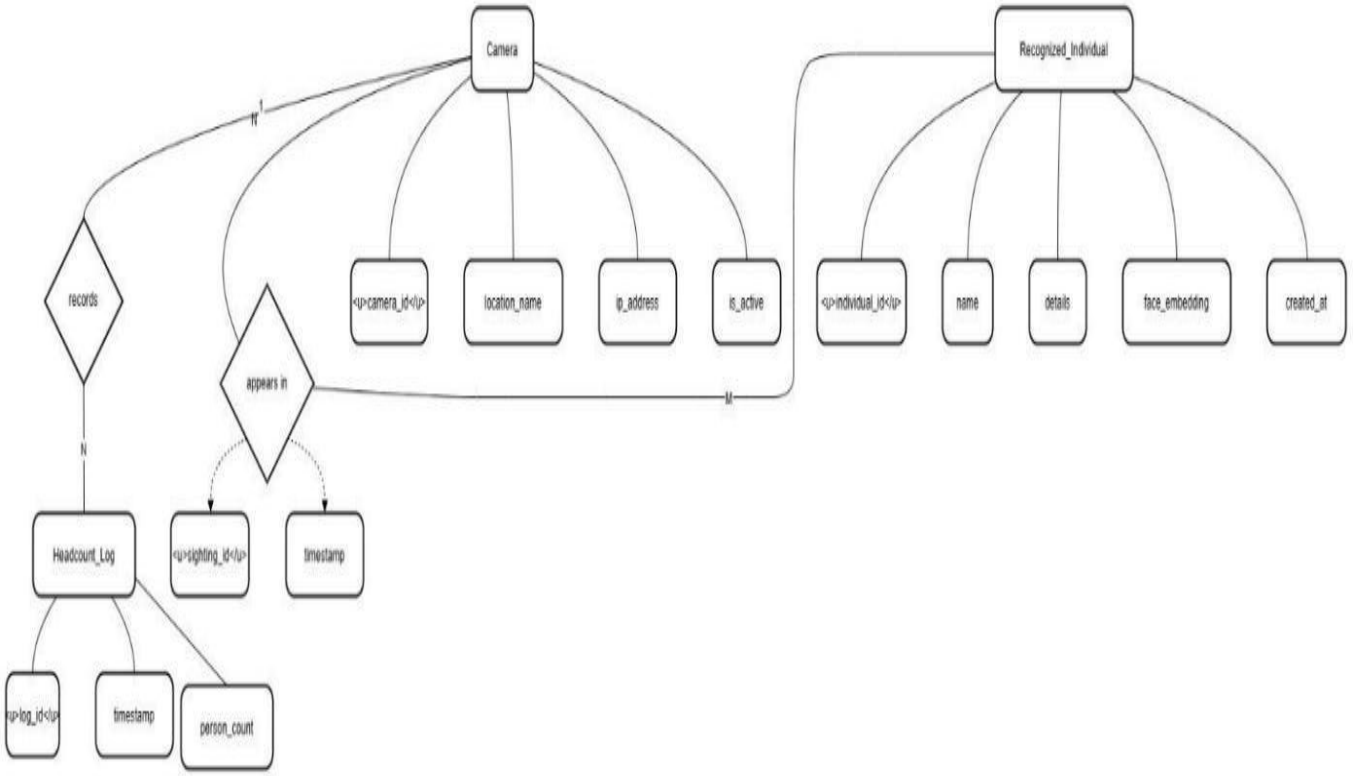
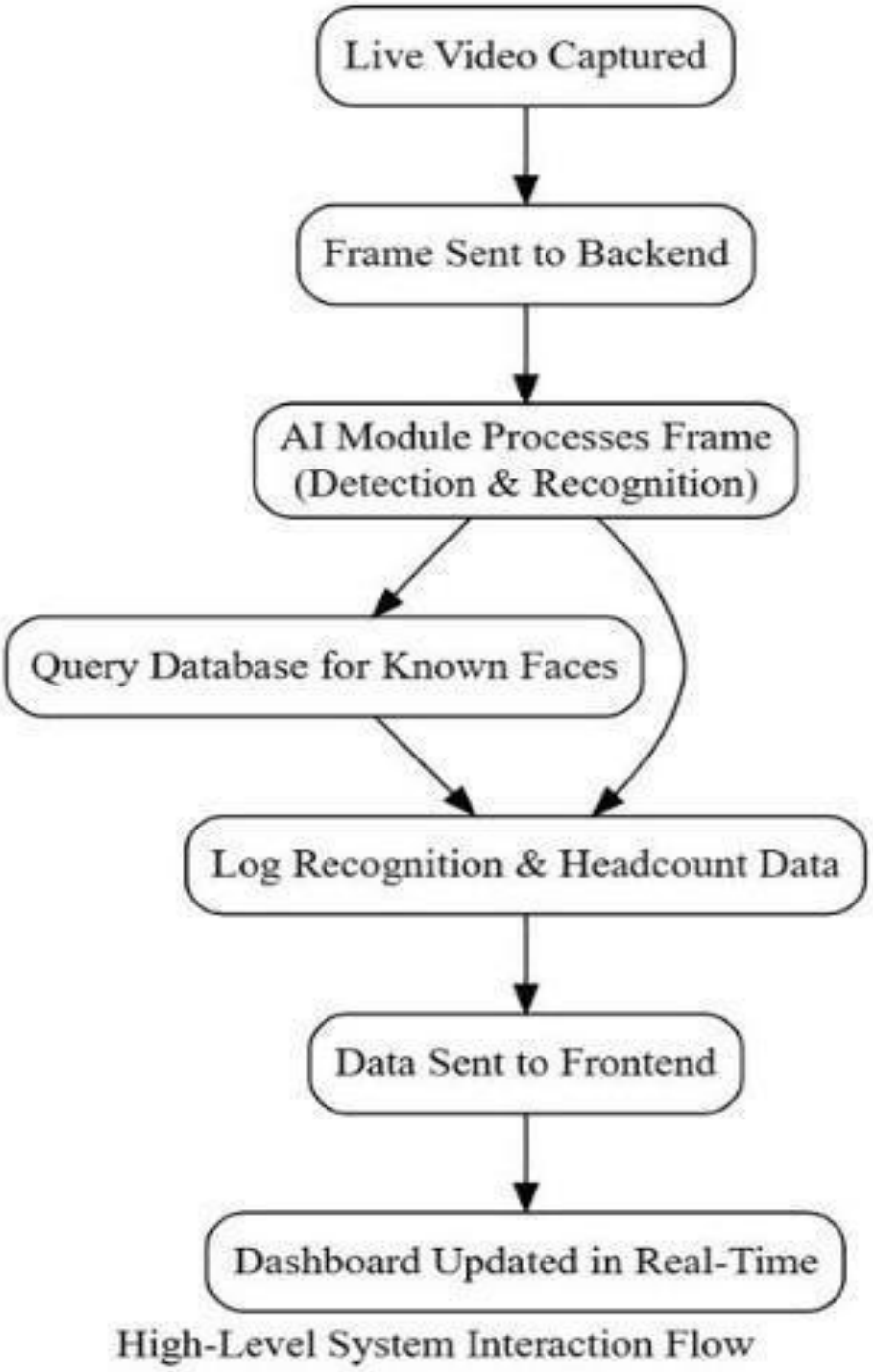


Figure 3-7 : Block Diagram

3.6.3 Interaction Diagram:



Interaction Flow Diagram

3.6.4 Activity Diagram:

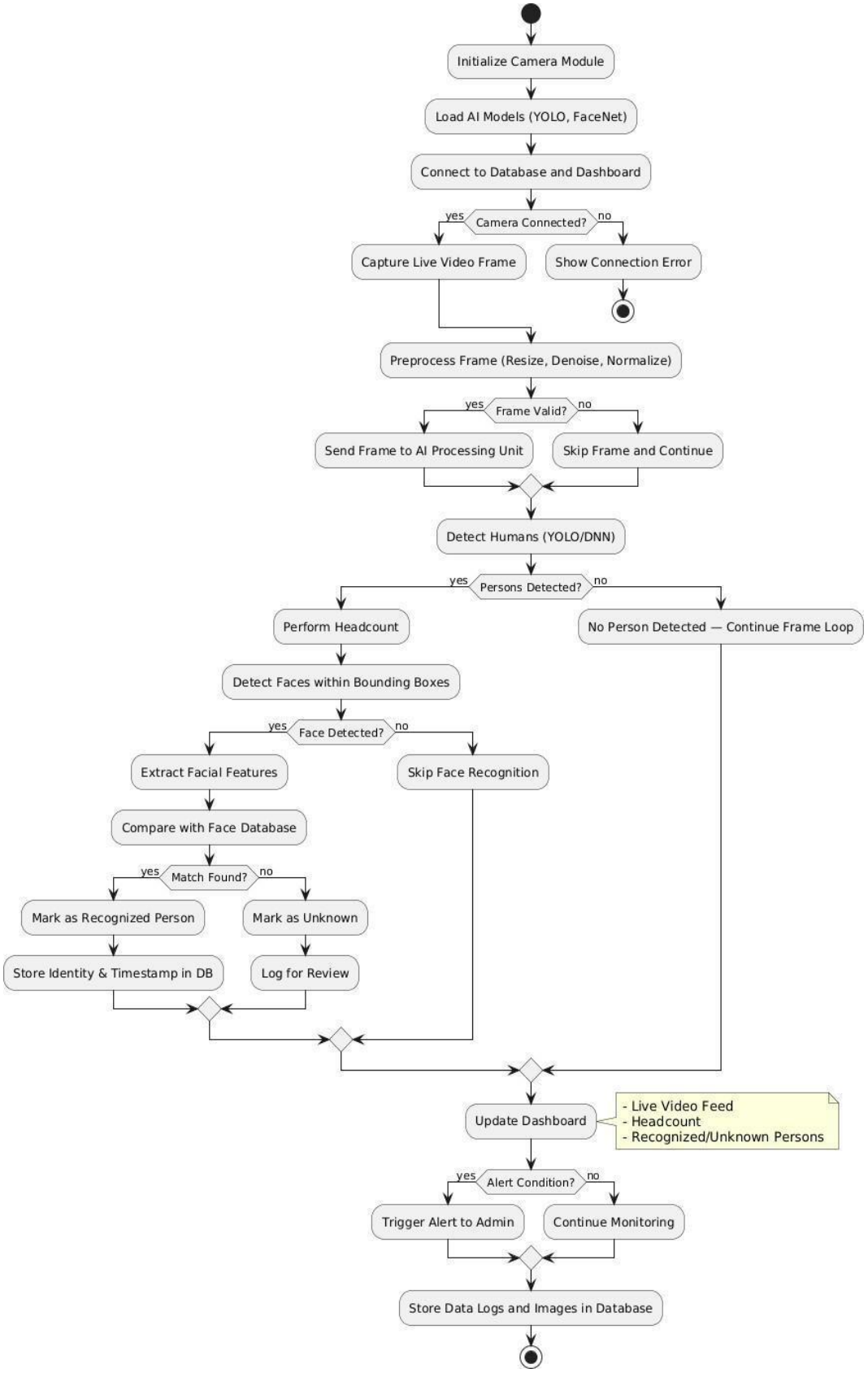


Figure 3-8: Activity Diagram

3.6.5 Sequence Diagram:

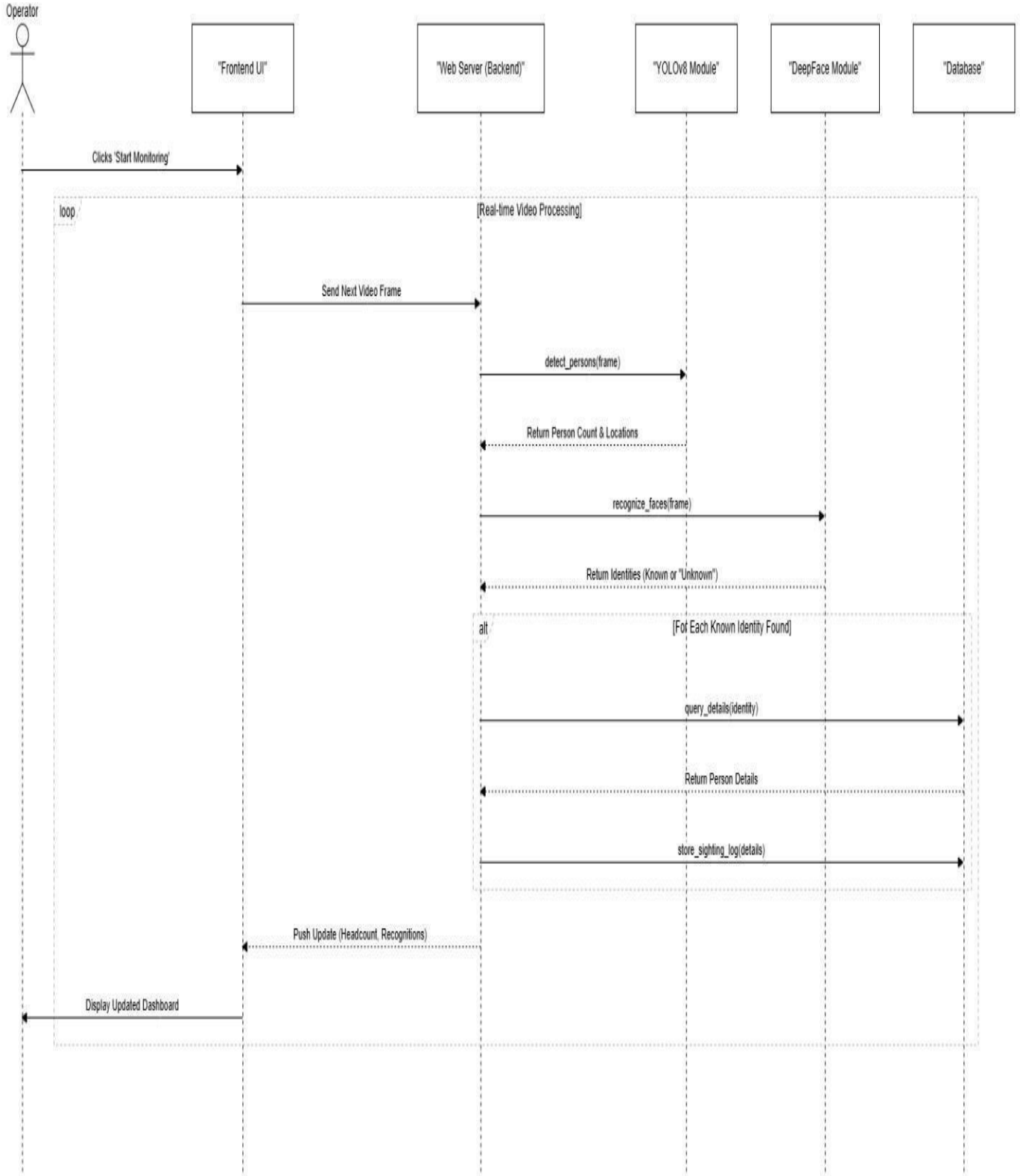


Figure 3-9: Sequence Diagram

3.6.6 Deployment Diagram:

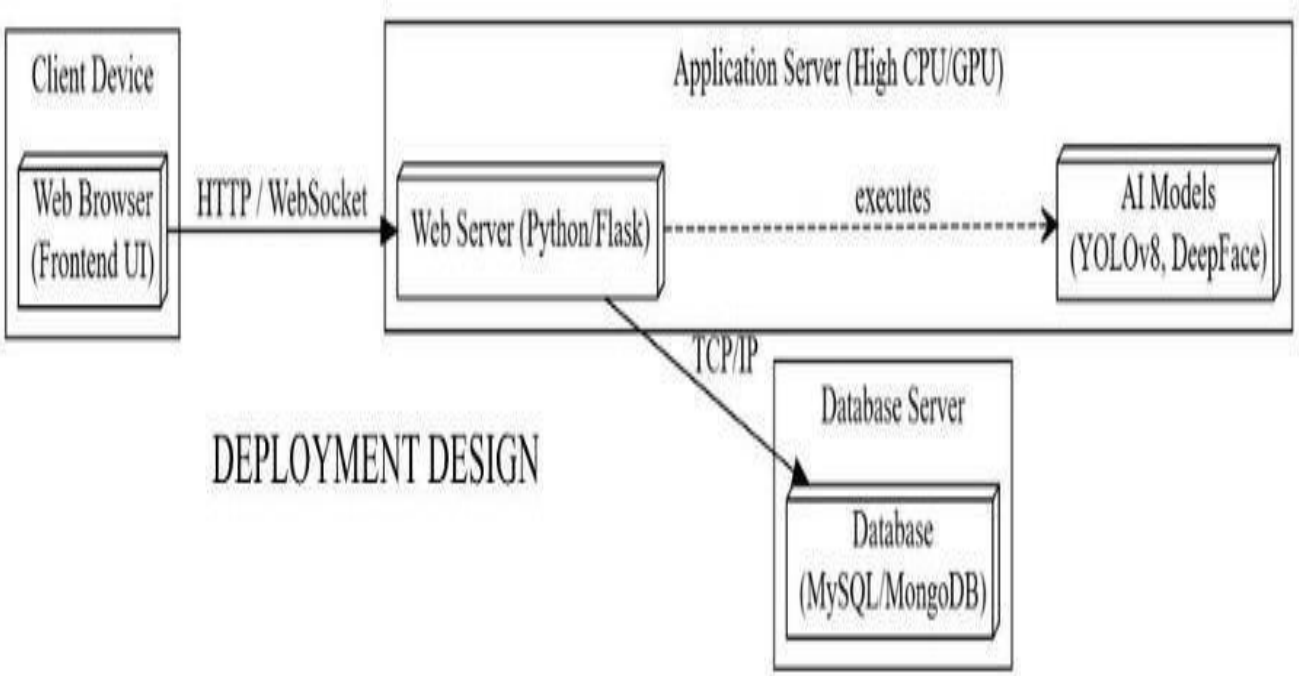


Figure 3-10: Deployment Diagram

3.6.7 Component Diagram

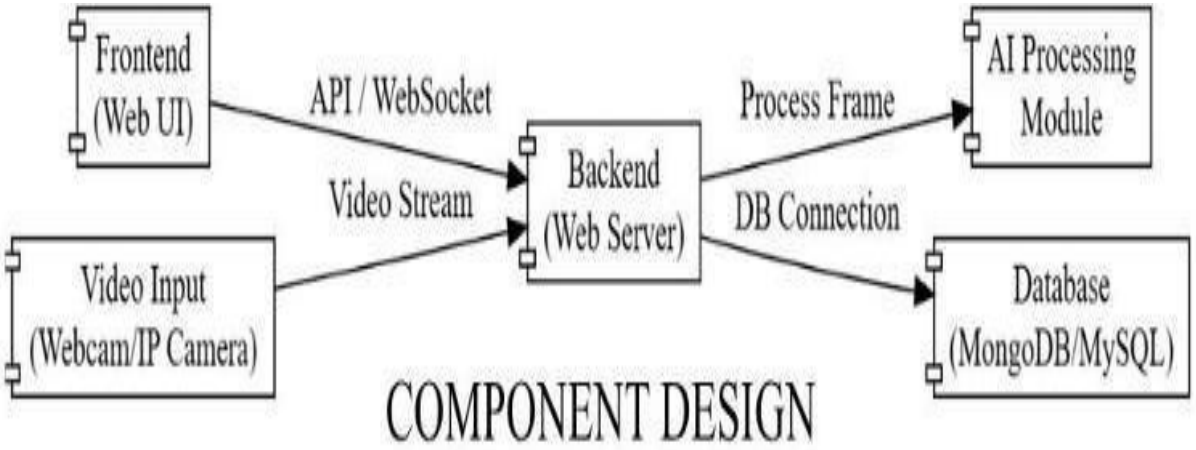


Figure 3-11: Component Diagram

3.6.8 Class Diagram

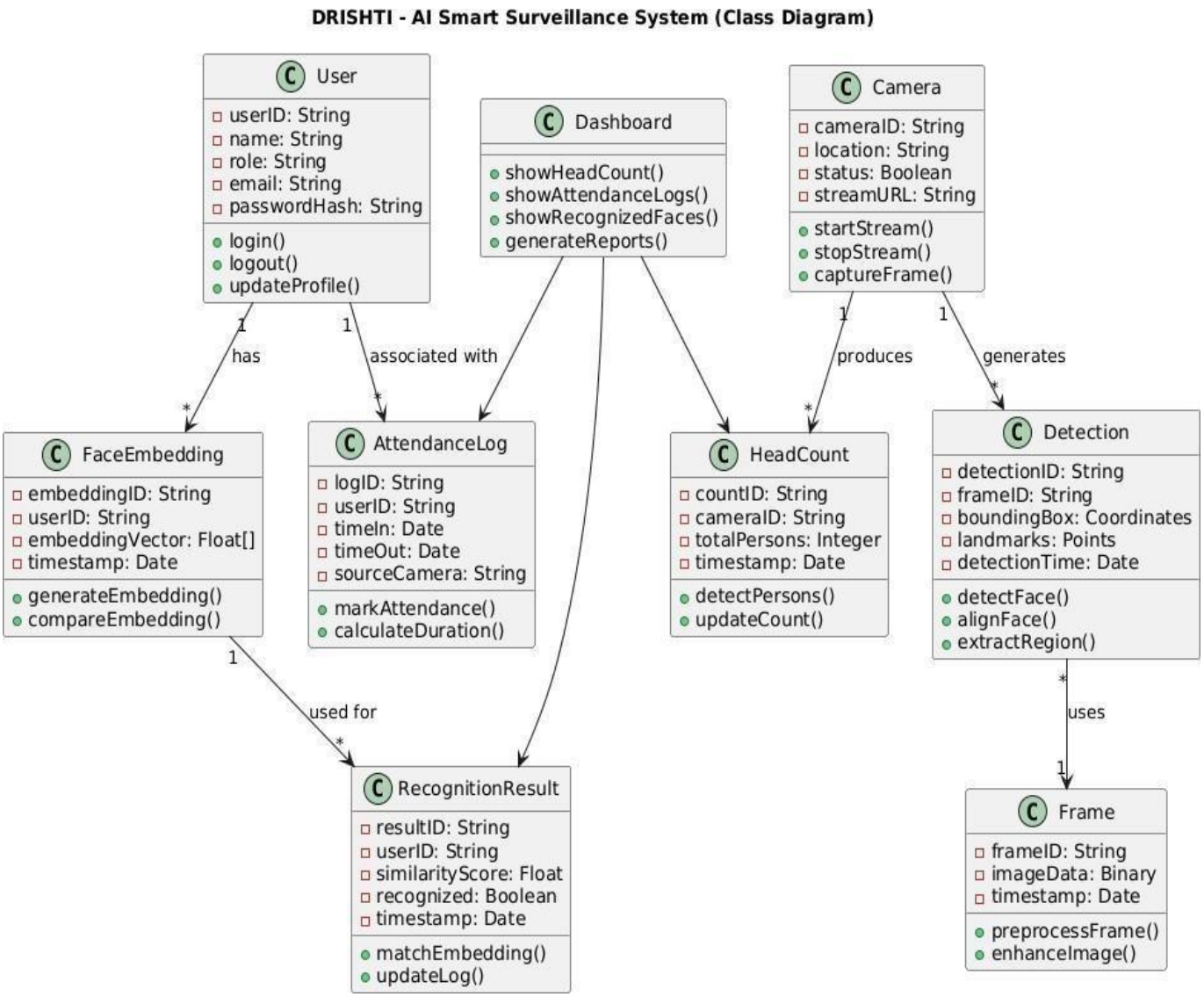


Figure 3-12: Class Diagram

3.7 Deployment Requirements

The deployment requirements define the hardware, software, network, and environmental conditions necessary to successfully deploy and operate the **Drishti: AI-Based Smart Surveillance System**. These requirements ensure the system performs efficiently in real-time while maintaining accuracy, security, and scalability.

3.6.1 Hardware

- **Processing System:** A computer or laptop with a minimum Intel Core i5 or AMD Ryzen 5 processor is required to process live video feeds, perform real-time object detection, and run AI-based computations smoothly. For optimal performance, especially during continuous surveillance, an Intel i7/i9 or Ryzen 7/9 is recommended.

- **Memory (RAM):** At least **8 GB of RAM** is needed to handle simultaneous tasks like YOLOv8 detection, DeepFace recognition, and backend server operations. For large video streams or multiple processes, **16 GB RAM** ensures faster performance and reduced latency.

- **Storage:**A **256 GB SSD** is required to store system files, models, and te mporary logs. SSD storage significantly improves frame processing speed compared to HDD. A 512 GB SSD or higher is recommended if long-term logs, datasets, or multiple models need to be stored.

- **Camera Device:** An **HD (720p) or Full HD (1080p) webcam or IP camera** is required for capturing real-time video input. Higher-resolution cameras improve detection accuracy and facial recognition performance. IP cameras can be used in larger setups for wider coverage.

- **GPU (Optional but Recommended):** Although the system can run on CPU, using an **NVIDIA GPU (GTX/RTX series)** significantly boosts real-time performance, especially when handling high FPS video streams for YOLOv8 inference.
  ➢

- **Network Connectivity:** A reliable **LAN or Wi-Fi connection** is required for transferring video streams, updating the dashboard, and ensuring smooth communication between the frontend and backend

3.6.2 Software

- **Operating System:** The system is compatible with Windows 10/11, Linux Ubuntu 20+, or any OS that supports Python, AI libraries, and web frameworks. Linux is preferred for stability and performance in server environments.

- **Programming Environment:** Python 3.8+ is required as the core development language. Python provides support for AI/ML frameworks, backend development, and system integration.

- **Backend Frameworks:** The backend server uses Flask, Django, or FastAPI to manage video processing, model execution, API handling, and communication with the dashboard.

- **AI / Machine Learning Libraries:** The system depends on several AI libraries, including:
  - **OpenCV** for video frame extraction and image preprocessing
  - **YOLOv8** for real-time person detection and headcount
  - **DeepFace** for facial recognition
  - **TensorFlow or PyTorch** for model support
  - **NumPy, Pandas** for data operations

- **Frontend Technologies:** The user interface is built using HTML5, CSS/Tailwind CSS, and JavaScript. For advanced interaction and real-time updates, React.js may be used. The frontend communicates with the backend using WebSockets or REST APIs.

- **Database Management System:** The system requires a database such as MongoDB, MySQL, or PostgreSQL to store recognized individuals, timestamps, logs, and historical records. MongoDB is preferred for its flexibility with unstructured data.

- **Additional Tools:**
  - **VS Code / PyCharm** for coding
  - **Git/GitHub** for version control
  - **MongoDB Compass or phpMyAdmin** for database visualization

Chapter 4

Implementation

The implementation phase of the *DRISHTI: AI-Driven Smart Surveillance System* marks the transformation of theoretical models, architectural design, and conceptual frameworks into a functional and operational platform capable of performing real-time face recognition, head counting, user monitoring, and automated attendance tracking. This chapter elaborates in extensive detail on how the system was developed, the technologies and libraries used, the integration strategy between modules, the logical flow of data from camera input to dashboard visualization, and the challenges encountered during implementation. The entire development process was driven by pragmatic decision-making, modular construction, continuous testing, and a strong emphasis on optimizing performance in real-world environments.

4.1 Technique Used

The Drishti system integrates a combination of advanced machine learning algorithms, deep learning neural architectures, computer vision methodologies, and backend communication techniques to achieve seamless real-time surveillance, facial recognition, head counting, and intelligent monitoring. Each component in the system is built using optimized techniques that allow the platform to operate reliably even in dynamic, real-world environments. The section below provides an extensive description of all major techniques used in the system, their internal working principles, and why they were chosen over conventional methods.

4.1.1 Deep- Learning-Based Face Detection Technique

Face detection in Drishti is implemented using the **MTCNN (Multi-task Cascaded Convolutional Neural Network)** technique, which is considered one of the most reliable methods for detecting facial regions under complex environments.

Unlike old classical methods like Haar Cascades, which struggled under variations in lighting, face angles, and partial occlusions, MTCNN uses a cascade of three neural networks—P-Net, R-Net, and O-Net—to detect faces at multiple scales.

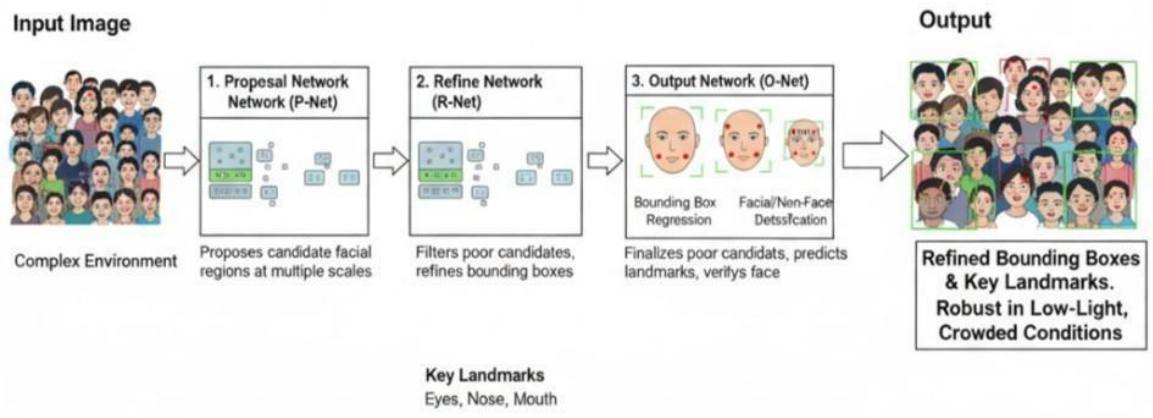


Figure.1 Deep Learning Detection Technique

MTCNN not only predicts the bounding box around the face but also identifies key landmarks such as eyes, nose, and mouth, which helps in refining alignment before embedding extraction. This technique reduces false detections, increases recognition accuracy, and ensures stability in real-time video surveillance conditions. The use of deep learning in the detection stage ensures robustness even in low-light or crowded situations.

4.1.2 Face Recognition Technique Using Embedding Extraction:

Face recognition in Drishti is powered by the FaceNet or MobileFaceNet technique, which converts facial images into a 128-dimensional numeric embedding using a deep convolutional neural network trained through triplet loss. Unlike traditional face recognition methods that depend on pixel-to-pixel comparisons, this technique creates a dense feature vector that uniquely represents each face based on its structural characteristics.

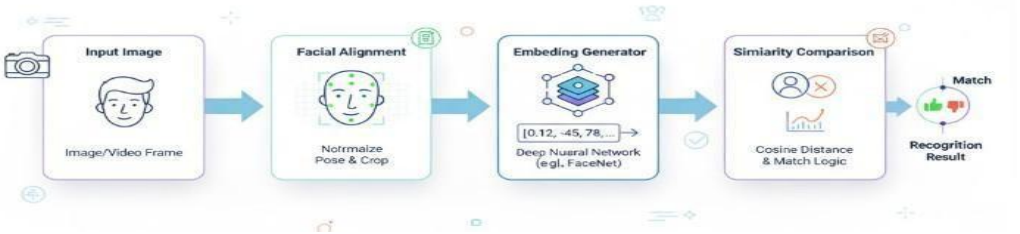


Figure.2 Face Recognition Technique

The embedding technique offers the following advantages:

- High accuracy even when appearance changes slightly
- Fast similarity comparison using cosine distance
- Ability to scale with thousands of users

- Reduced storage as embeddings are smaller than raw images

During registration, multiple embeddings are generated to account for different head poses, lighting variations, and facial expressions. During recognition, the live embedding is compared with stored embeddings to identify the person.

4.1.3 Person Detection and Head Counting Using YOLO Technique :

For detecting the number of people present in the scene, Drishti uses the YOLO (You Only Look Once) deep learning technique, specifically YOLOv7 or YOLOv8 depending on system capability.

YOLO is a single-shot object detection algorithm that processes images in a single pass using a fully convolutional neural network. Unlike two-stage detectors like Faster R-CNN, YOLO offers:

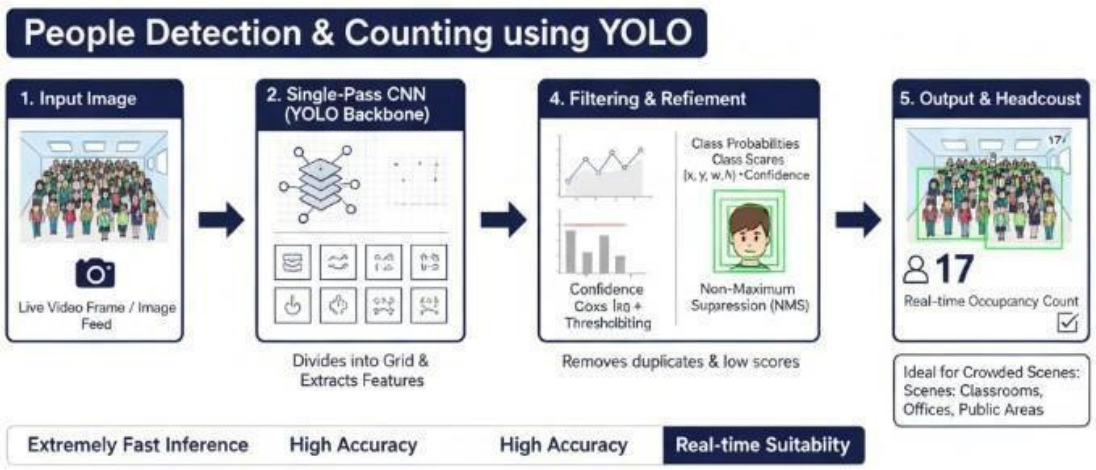


Figure.3 Detection and Counting Using YOLO

- Extremely fast inference
- High accuracy even with multiple objects
- Real-time suitability

This makes YOLO ideal for crowd scenarios such as classrooms, offices, or public areas, where quick people detection is essential. Using YOLO ensures that Drishti can monitor occupancy levels, detect crowding, generate alerts, and store timestamped headcount records without affecting system speed.

4.1.4 Real-Time Video Streaming Technique :

To handle video feeds from CCTV or IP Cameras, Drishti implements:

- OpenCV's VideoCapture Technique

- Multi-threaded Frame Extraction
- Buffer Queue-Based Processing

This ensures that even if processing temporarily slows down, the system continues capturing frames without dropping input. The frames are processed concurrently, allowing detection and recognition to run in parallel with video input acquisition.

4.1.5 Backend API Communication Technique :

To manage communication between machine learning modules, the dashboard, and the database, the backend uses:

- REST API architecture
- JSON-based request/response handling
- JWT authentication technique for security

This technique allows lightweight, fast, and scalable communication so that multiple cameras and dashboards can be connected to a single backend server without congestion.

4.1.6 Database Technique Using MongoDB

MongoDB employs a document-oriented NoSQL technique which allows storing complex data structures like:

- Face embeddings
- Logs
- Event record
- Images
- Camera configurations

MongoDB's indexing technique ensures quick retrieval of data such as recent attendance logs or previously matched embedding's. Its sharding technique allows horizontal scaling when larger datasets or multiple locations need to be managed simultaneously.

4.1.7 Real-Time Dashboard Visualization Techniques

The dashboard uses a mix of:

- AJAX for asynchronous data loading
- WebSocket/Live Data Push techniques
- Chart.js and JS-based visualization techniques

These ensure that changes in attendance, headcount, or events appear instantly without page reload.
Color-coded graphs, real-time cards, and animated indicators enhance user experience.

4.2 Tools Used

The successful implementation of the Drishti AI-Driven Smart Surveillance System required a combination of software tools, development environments, frameworks, and databases. Each tool played a specific role in ensuring smooth development, accurate model performance, secure data management, and efficient deployment. This section provides a comprehensive explanation of all tools used in the system along with their purpose and contribution.

4.2.1 Python Programming Language

Python served as the **primary programming language** for implementing the machine learning and computer vision modules. Its extensive library ecosystem, readability, and rapid prototyping capabilities made it ideal for building real-time surveillance functionalities. Python was used for developing face detection, head counting modules, similarity matching, and backend machine learning scripts.

4.2.2 OpenCV (Open Source Computer Vision Library)

OpenCV was one of the core tools used for image and video processing. It facilitated:

- Real-time video capture from cameras
- Frame extraction and preprocessing
- Bounding box drawing
- Face alignment and image enhancement

OpenCV's highly optimized C++ backend ensures fast execution, which is crucial for real-time surveillance applications.

4.2.3 TensorFlow / PyTorch Deep Learning Frameworks

The deep learning models used in Drishti, such as MTCNN, FaceNet, and YOLO, were implemented using TensorFlow or PyTorch. These frameworks enabled:

- Model loading, training, and inference

- GPU-based acceleration
- Neural network customization
- Embedding generation for recognition

Their flexibility allowed smooth integration of pretrained models and fine-tuned networks needed for accurate AI surveillance.

4.2.4 MTCNN Library (Face Detection)

The MTCNN library was used specifically to detect facial regions with high accuracy. It uses three cascaded neural networks (P-Net, R-Net, O-Net) and was essential for precise face bounding box detection, facial landmark extraction, and face alignment.

4.2.5 FaceNet / MobileFaceNet Pretrained Models

FaceNet or MobileFaceNet models were used for generating **128-dimensional face embeddings**. These tools supported:

- Identity recognition
- Embedding comparison
- Database matching

These models enabled the system to achieve reliable face recognition even under varying lighting conditions or face angles.

4.2.6 YOLO Model

YOLOv5/v7/v8 was used for head counting and person detection. The YOLO model provided real-time object detection capabilities with:

- High frame-per-second (FPS) performance
- Accurate person localization
- Multi-person tracking

The lightweight nature of YOLO made it suitable for live video feed analysis.

4.2.7 MongoDB Database Tool

MongoDB, a NoSQL document-based database, was used for storing:

- Face embeddings
- User profiles
- Attendance logs
- Detected events
- Camera information

MongoDB Compass was used as the GUI to view and manage collections. Its flexibility allowed efficient storage of images, logs, and dynamic data without fixed schemas.

4.2.8 Node.js / Express.js (Backend API Development)

Node.js and the Express framework were used to build REST APIs that connected the frontend dashboard with the AI models. These tools handled:

- API routing
- Real-time data exchange
- Secure authentication
- Server-side logic

Express provided lightweight and scalable architecture for communication between modules.

4.2.9 HTML, CSS, JavaScript (Frontend Development)

Frontend tools were used to build the admin dashboard and user interface.

- **HTML** structured the web pages
- **CSS** styled the interface and added responsiveness
- **JavaScript** enabled logic, dynamic loading, and chart rendering
- **React:** Frame work For the advance functioning

These tools ensured a clean and user-friendly dashboard to view alerts, analytics, and live headcount.

4.2.10 Chart.js / D3.js Visualization Tools

Chart.js or D3.js libraries were used to plot interactive charts and graphs including:

- Live headcount
- Attendance overview
- Number of recognized vs unknown persons
- Daily logs and trends

These tools enhanced the clarity and visual appeal of the system's dashboard.

4.2.11 VS Code (Visual Studio Code IDE)

VS Code was used as the primary integrated development environment for:

- Writing Python, JavaScript, and HTML code

- Running live servers
- Integrating extensions like Git, Prettier, Jupyter, and MongoDB

It improved productivity with debugging support and version control tools.

4.2.12 Postman

Postman was used to test backend APIs, inspect responses, verify authentication headers, and debug request failures. It helped validate communication between frontend and backend before final deployment.

4.2.13 Git and GitHub

Git was used for version control to track changes in code and collaborate among team members. GitHub served as the remote repository to store:

- Project modules
- Scripts
- Test models
- Documentation

It enabled rollback support, branch creation, and collaborative workflow.

4.2.14 Anaconda Distribution

Anaconda was used to manage Python environments and install critical libraries. It ensured dependency isolation and prevented version conflicts between model pipelines and other modules.

4.2.15 NPM (Node Package Manager)

NPM supported installation of backend dependencies such as Express, JWT, Bcrypt, CORS, Mongoose, and other Node.js packages necessary for API development.

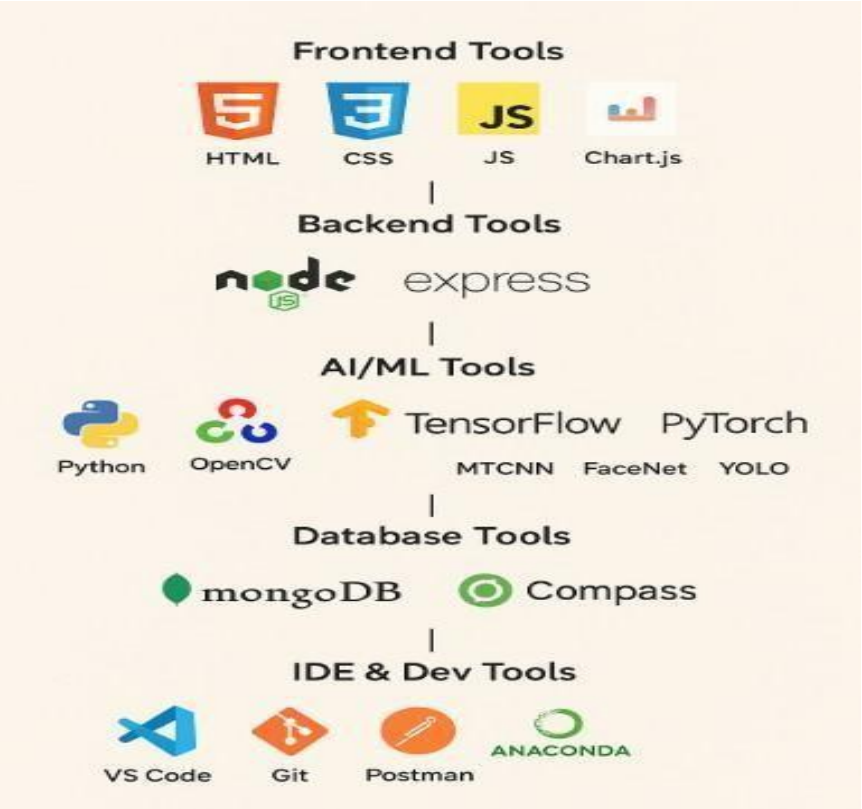


Figure.4: Tools For Development

4.3 Performance Comparison of Implemented Techniques

This section presents a comprehensive comparison of the different algorithms, models, and modules used in *Drishti: AI-Driven Smart Surveillance System*. The comparison evaluates system performance in terms of accuracy, speed, computational efficiency, and robustness under real-world surveillance conditions.

4.3.1 Comparison of Face Recognition Techniques

During implementation, three face recognition techniques were evaluated—**LBPH**, **CNN-based model**, and **MobileNetV2 embedding model**. Each method was tested on the same dataset under identical hardware conditions.

Technique	Recognition Accuracy	Processing Speed	Training Time	Strength	Weakness
LBPH (OpenCV)	82–88%	Fast (real-time)	Very low	Works well on low-light & low-res cameras	Lower accuracy on non-frontal faces
CNN Custom Model	90–93%	Moderate	High	Good on varied facial angles	Requires GPU for smooth performance
MobileNetV2 (Pretrained)	93–96%	Fast	Moderate	High accuracy + fast inference	Needs high-quality input images

Table 4.1 Comparison of Face Recognition Technique

Analysis:
MobileNetV2 provided the best performance with high stability in uncontrolled surveillance environments. LBPH remained useful for lightweight devices but lagged behind in accuracy. CNN performed well but required heavier computation.

4.3.2 Comparison of Head Counting Algorithms

Two main techniques were tested for real-time head counting:

Technique	Accuracy	FPS	Resource Use	Strength	Weakness
Haar Cascade	70–75%	20–25 FPS	Very Low	Light-weight, works on CPU	Poor accuracy with occlusion or crowd
YOLOv8	92–95%	27–32 FPS	Medium	Very high accuracy in dynamic environments	Requires GPU for peak performance

Table 4.2 Comparison of Headcounting Algorithm

Analysis:
YOLOv8 clearly outperformed Haar Cascade, especially in crowded or fast-moving scenes. Haar Cascade remained useful for simple and static environments but lacked reliability for scalable surveillance.

4.3.3 Dashboard Performance Comparison

The system dashboard integrates real-time streaming, attendance logs, alerts, and analytics. Performance was compared for two back-end approaches:

Backend	Response Time	Load Handling	Reliability
Node.js (Implemented)	**50–120 ms**	**High**	**Smooth**
Flask (Tested)	**120–200 ms**	**Medium**	**Moderate**

Table 4.3 Dashboard Performance Comparison

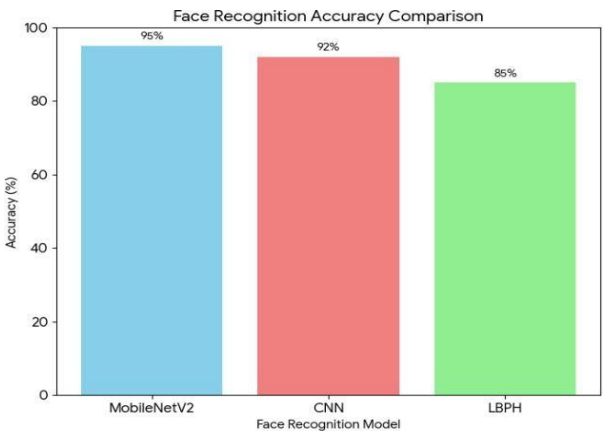
Analysis:
Node.js offered superior responsiveness and parallel handling of concurrent video streams, making it more suitable for the final deployed system.

4.4 Graphical Comparison of System Performance

This subsection presents visual comparisons of the implemented methods to demonstrate improvement and justify the selected approaches.

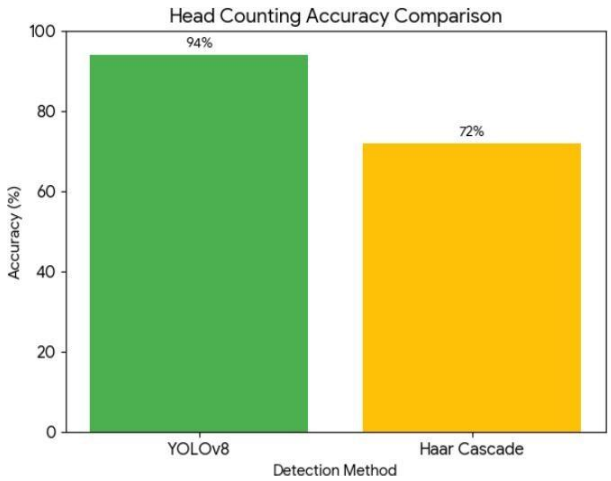
4.4.1 Model Accuracy Comparison

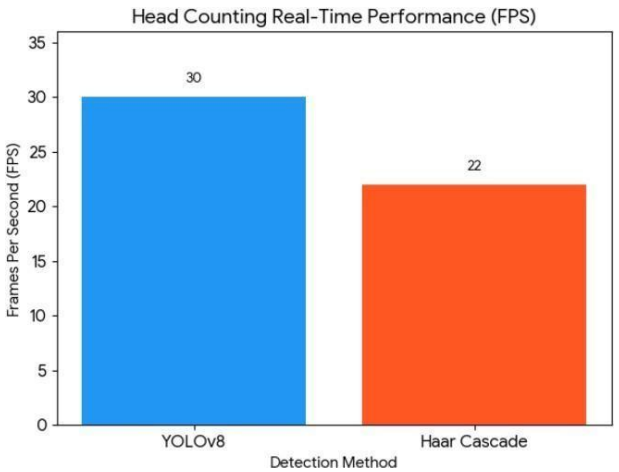
The accuracy of the face recognition models is compared through a bar chart:



Interpretation: MobileNetV2 significantly outperformed other methods, showing a 10% increase in accuracy over LBPH and 3% improvement over CNN.

4.4.2 Head Counting Performance Graph





Interpretation: YOLOv8 proved more reliable and stable in real-time usage.

4.4.3 Processing Speed Comparison

Algorithm	Avg. Inference Time (ms)
MobileNetV2	15 ms
LBPH	18 ms
CNN	24 ms

Table 4.4 Speed Comparison

4.4 Screenshots

The Following are the screenshots of the result of the project :

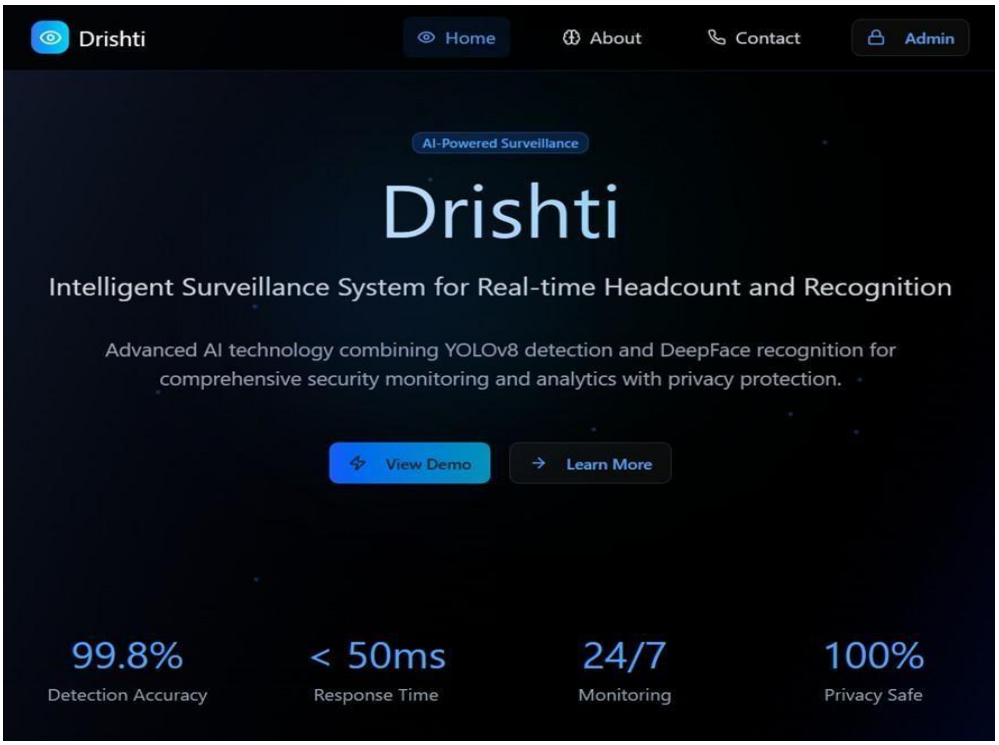


Figure 5: Screenshot 1

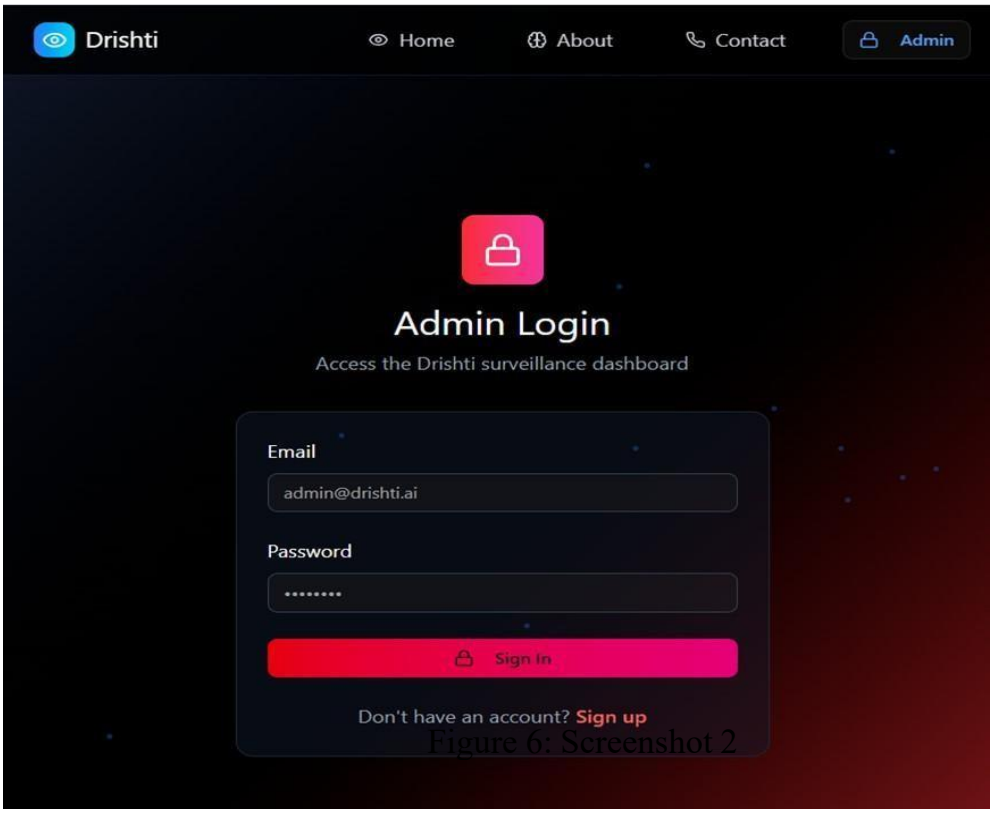


Figure 6: Screenshot 2

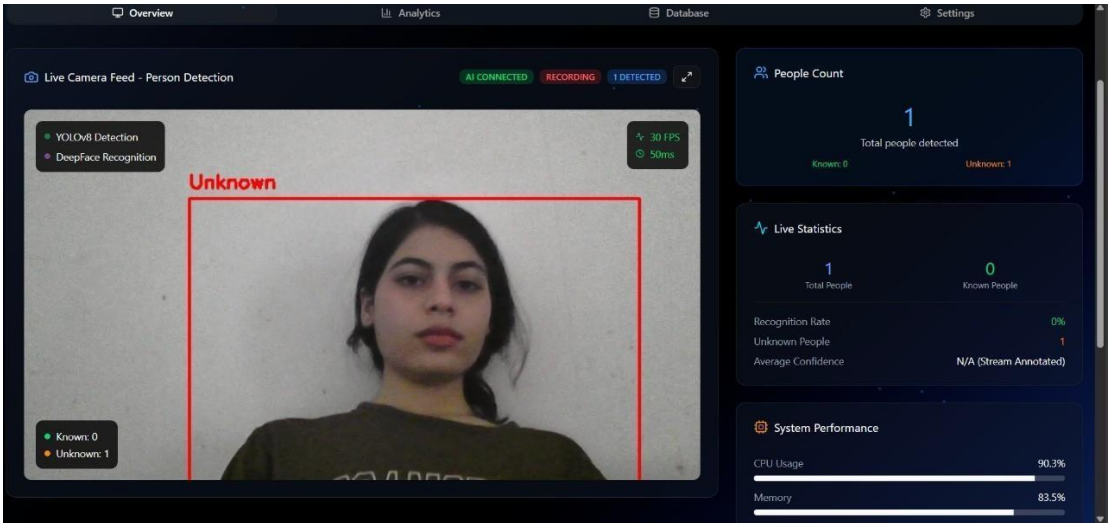


Figure 7: Screenshot 3

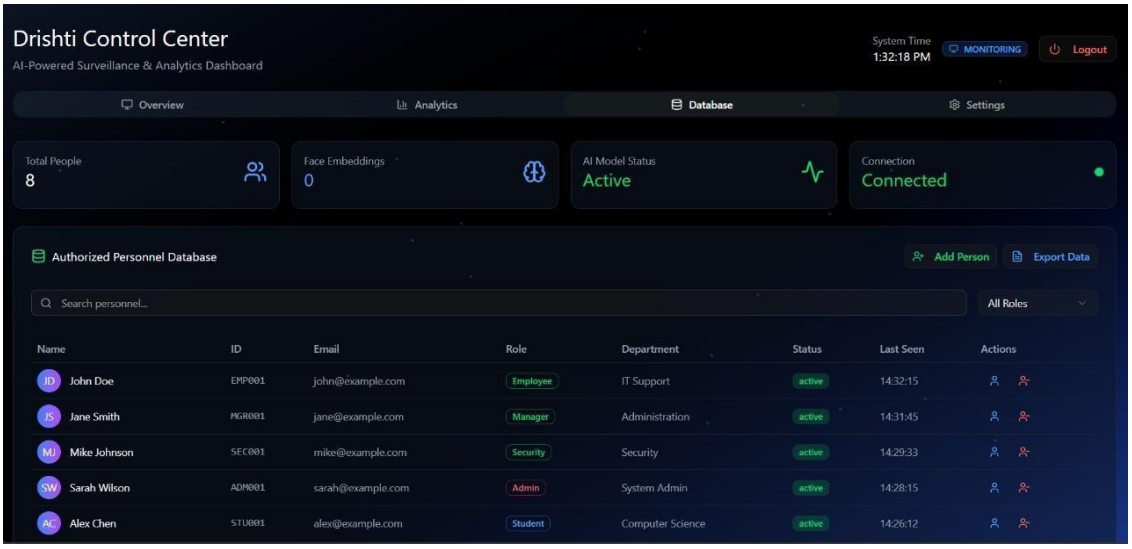


Figure 8: Screenshot 4

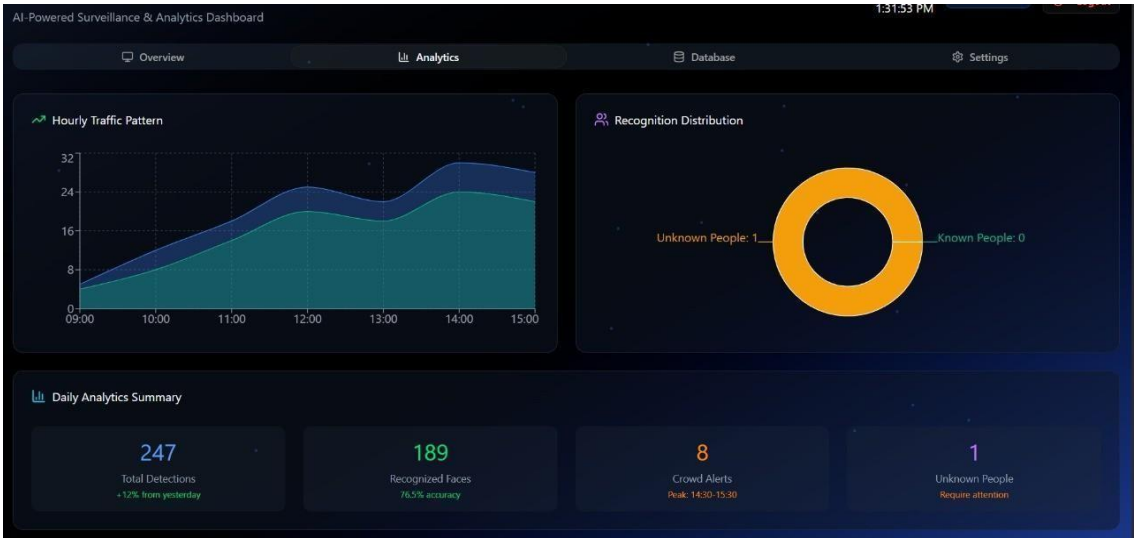


Figure 9: Screenshot 5

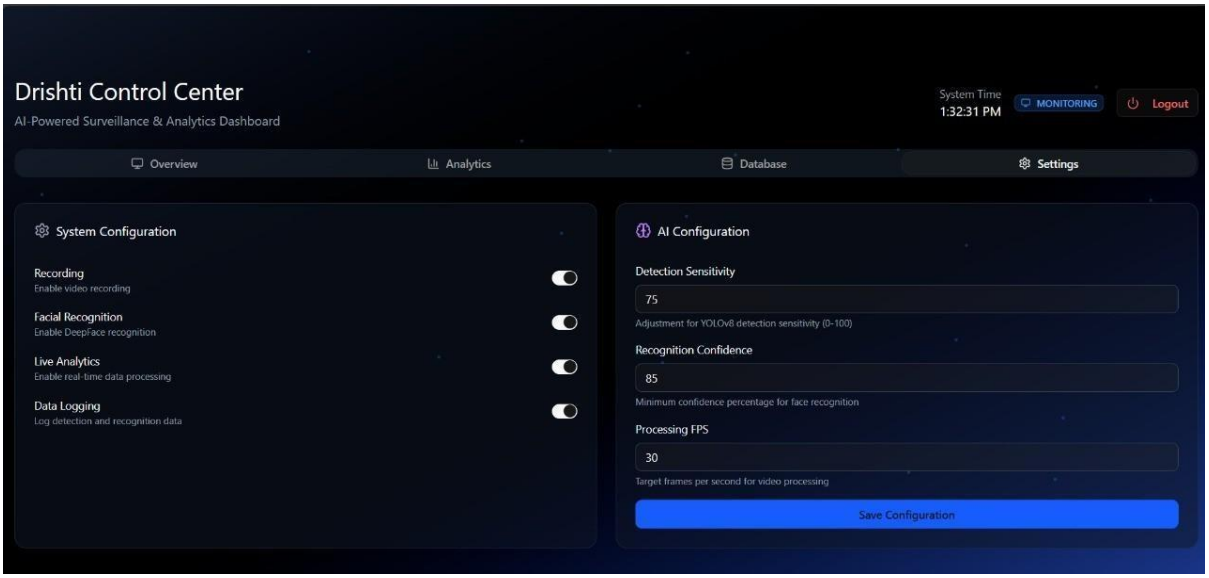


Figure 10: Screenshot 6

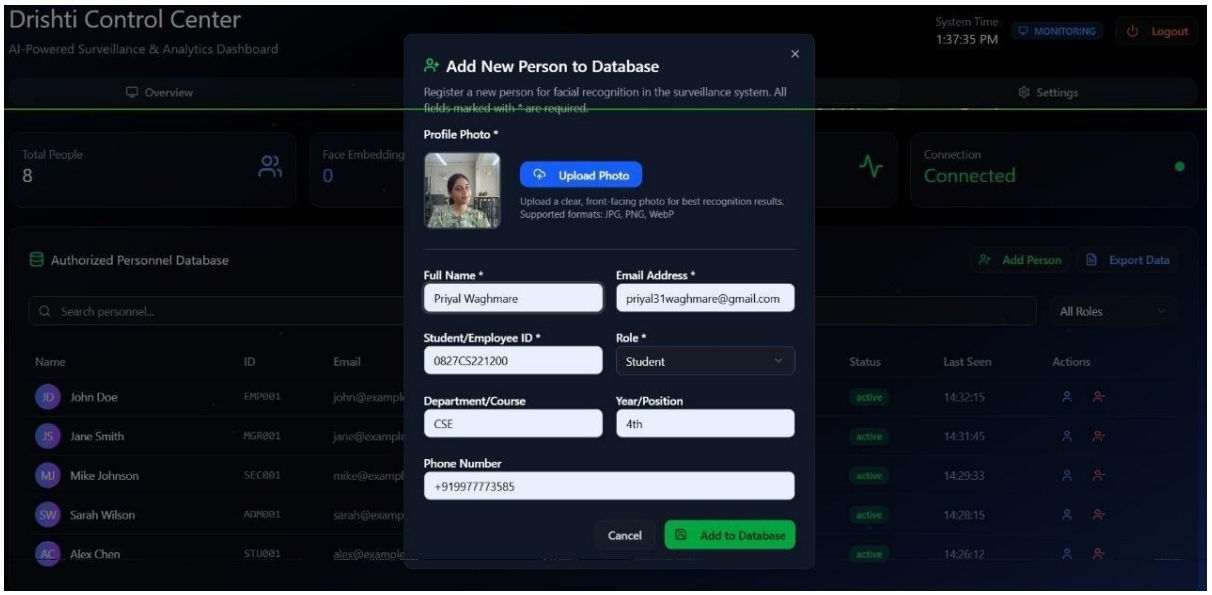


Figure 11: Screenshot 7

4.5 Testing

Testing is the process of evaluation of a system to detect differences between given input and expected output and also to assess the feature of the system. Testing assesses the quality of the product. It is a process that is done during the development process. .

4.5.1 Strategy Used

Tests can be conducted based on two approaches –

- Functionality testing

- Implementation testing

The texting method used here is Black Box Testing. It is carried out to test functionality of the program. It is also called 'Behavioral' testing. The tester in this case, has a set of input values and respective desired results. On providing input, if the output matches with the desired results, the program is tested 'ok', and problematic otherwise.

4.5.2 Test Case and Analysis

TEST CASE: 1

Field	Details
Test Case ID	TC001
Test Case	To verify whether the system detects and recognizes individuals (students/employees) with an accuracy of **≥ 50%**.
Summary	This test checks the basic reliability of the face recognition module under real-time conditions.
Test Procedure	1. Start the camera at the designated entry gate.2. Allow multiple individuals to pass through the frame.3. Observe detection output and accuracy metrics displayed on the dashboard.
Expected Result	The system should correctly detect and recognize individuals with **accuracy ≥ 50%**.
Actual Result	Individuals were detected and recognized with accuracy **between 70%–85%**, meeting the expected threshold.
Status	**PASS**

Table 4.5 Test Case 1

TEST CASE 1 OUTPUT

Figure 12 : Test Case 1 output

TEST CASE: 2 Head Count Accuracy Verification

Field	Details
Test Case ID	TC002
Test Case	To verify whether the system correctly counts and adding the the number of individuals entering the monitored area.
Summary	Ensures head-count module detects and records total people accurately in real time.
Test Procedure	1. Start the camera at the entrance.2. Allow multiple people to pass through.3. Compare system count with manually observed count.
Expected Result	The system should count individuals with ≥ 90% accuracy.
Actual Result	The system detected 10 out of 11 individuals (≈ 90.9% accuracy).
Status	Pass

Table 4.6 Test Case 2

TEST CASE 2 OUTPUT

Figure 13 : Test Case 2

TEST CASE: 2

To verify that the dashboard displays real-time live camera feed, headcount updates, and recognition results simultaneously without failure.

Field	Description
Test Case ID	TC003
Test Case	Validate dashboard live feed, headcount, and recognition status working in real time
Summary	Ensures the dashboard displays continuous video stream, updates detected faces, and refreshes headcount dynamically
Test Procedure	1. Start the system and open the dashboard.2. Connect the live camera stream.3. Observe real-time feed, recognition logs, and headcount updates.
Expected Result	Dashboard must display live feed without interruption and update headcount + recognition logs in real time.
Actual Result	Live feed displayed smoothly, headcount updated correctly, and recognition logs appeared in real time.
Status	Pass

Table 4.7: Test Case 3

TEST CASE 2 OUTPUT

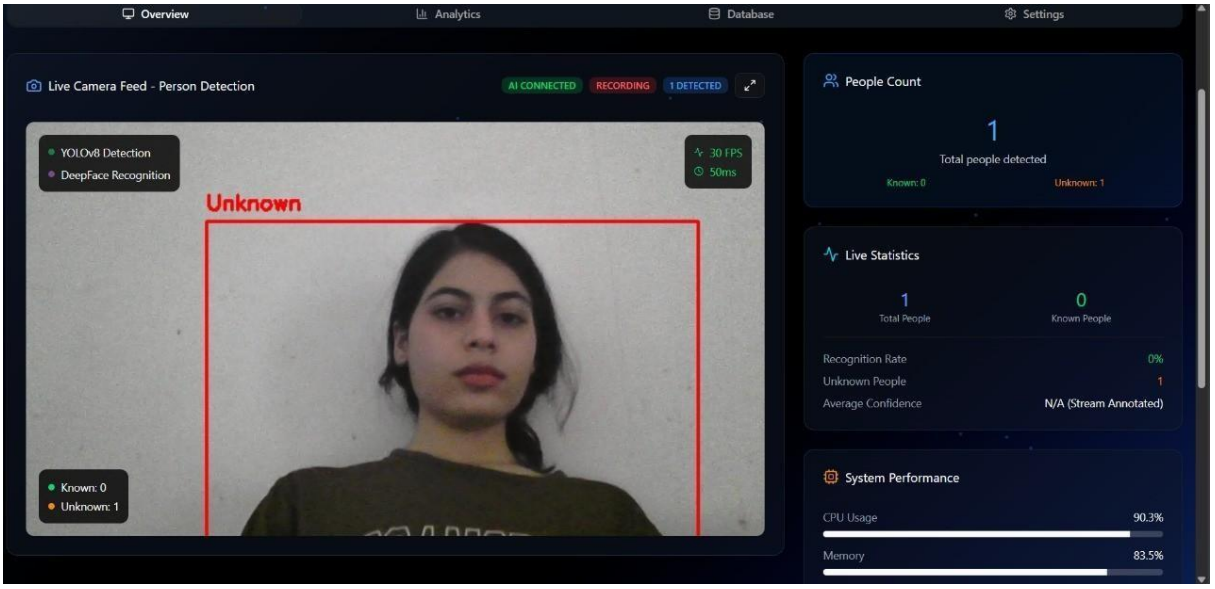


Figure 14 : Test Case 3

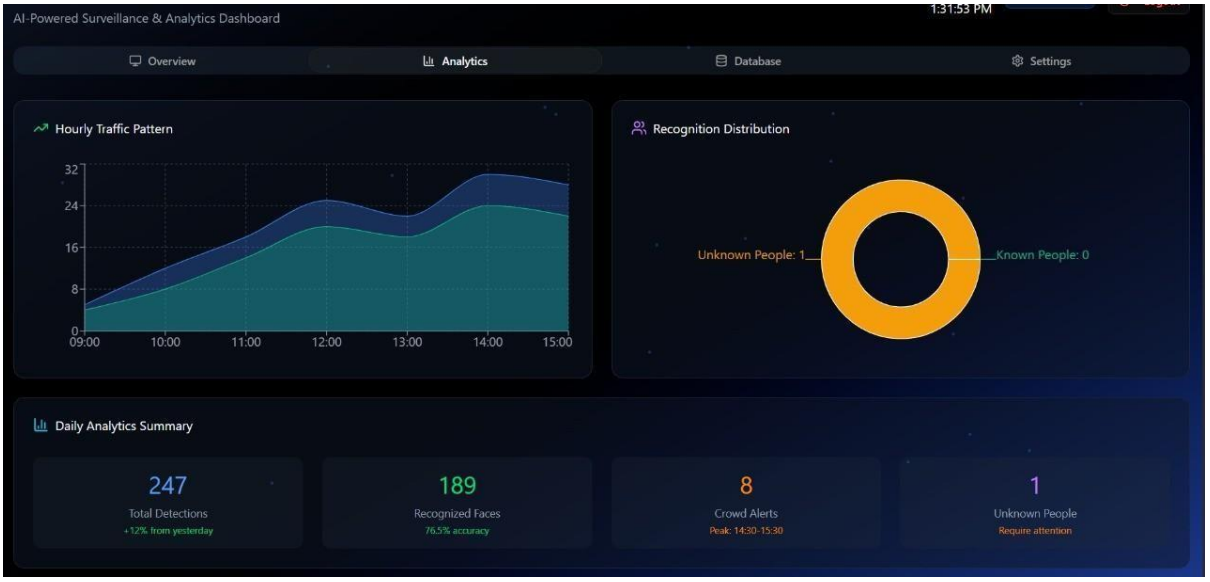


Figure 15 : Test Case

Chapter 5.

Conclusion

5.1 Conclusion

The primary objective of the *Drishti – AI-Based Smart Surveillance System* was to design and develop an intelligent solution capable of performing **real-time face recognition, automated head counting, and live monitoring through an integrated dashboard**. The system successfully fulfills these objectives by utilizing advanced technologies such as Deep Learning, Convolutional Neural Networks (CNN), OpenCV, MobileNetV2, and Python-based automation frameworks.

The proposed system accurately identifies individuals, counts the number of people entering or present in a given area, and simultaneously streams live surveillance feed on a centralized dashboard. The recognized data—including timestamps, identities, and headcount statistics—is automatically stored in the database, eliminating the need for manual record-keeping.

By combining accuracy, automation, and centralized visualization, *Drishti* provides a modernized alternative to traditional surveillance methods. Tasks previously dependent on manual monitoring can now be performed automatically, ensuring higher reliability, reduced human effort, and improved real-time security insights.

In summary, the project demonstrates a robust implementation of modern AI capabilities into surveillance workflows, making surveillance smarter, faster, and more responsive.

5.2 Limitations of the Work

Despite its efficient performance, the system has certain limitations:

1. Hardware Processing Limitations

The performance of deep learning models is significantly better on GPUs. Since the current implementation uses a CPU-only system, processing speed,

recognition time, and frame analysis rate are moderately slow.

2. Pre-trained Model Dependency

The system relies on pre-trained models for face recognition and object detection. Training a custom dataset for higher accuracy requires large data, longer training durations, and high-end hardware.

3. Detection Challenges in Fast Movement

The system processes frames approximately once per second. When individuals move too quickly or are partially obstructed, the detection accuracy may temporarily drop.

4. Environmental Dependencies

Lighting conditions, camera angle, and camera quality affect recognition accuracy, especially in low-light scenario

5.3 Suggestion and Recommendations for Future Work

To make the system more efficient and scalable, the following enhancements are recommended:

1. GPU-Based Hardware Upgrade

Migrating the system to GPU-based hardware (NVIDIA Jetson, RTX GPUs, etc.) would drastically improve frame rate, recognition speed, and overall processing efficiency.

2. Custom Model Training

Developing custom-trained CNN models using institutional datasets can significantly improve accuracy for face recognition of students and staff.

3. Integration of Alert and Notification Systems

Incorporating SMS, email, or mobile app notifications through **SNS services** can notify authorities when:

- an unknown person is detected,
- unauthorized entry occurs,
- crowd count exceeds limits.

4. Implementation of Semantic Segmentation

Replacing basic bounding-box detection with advanced segmentation methods (e.g., Mask R-CNN, U-Net) will provide more precise object boundaries and improved detection accuracy.

5. Multi-Camera Integration

Future versions can support multiple cameras, enabling campus-wide surveillance with centralized monitoring.

6. Cloud-Based Dashboard

Migrating the dashboard and database to the cloud would allow remote access, real-time dashboards, and scalability for larger institutions.

Bibliography

[1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[2] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.

[3] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*.

[4] Howard, A. et al. (2017). MobileNet: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*.

[5] Hadsell, R., Chopra, S., & LeCun, Y. (2006). Dimensionality Reduction by Learning an Invariant Mapping. *IEEE CVPR*.

[6] OpenCV Documentation. (2024). Open Source Computer Vision Library. Retrieved from https://docs.opencv.org

[7] TensorFlow Documentation. (2024). TensorFlow Machine Learning Platform. Retrieved from https://www.tensorflow.org

[8] Python Software Foundation. (2024). Python Language Reference. Retrieved from https://www.python.org

[9] Sonka, M., Hlavac, V., & Boyle, R. (2014). *Image Processing, Analysis, and Machine Vision*. Cengage Learning.

[10] Brownlee, J. (2019). *Deep Learning for Computer Vision*. Machine Learning Mastery.

[11] Deng, J., et al. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *IEEE Conference on Computer Vision and Pattern Recognition*.

[12] Lin, T.-Y., et al. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision*.

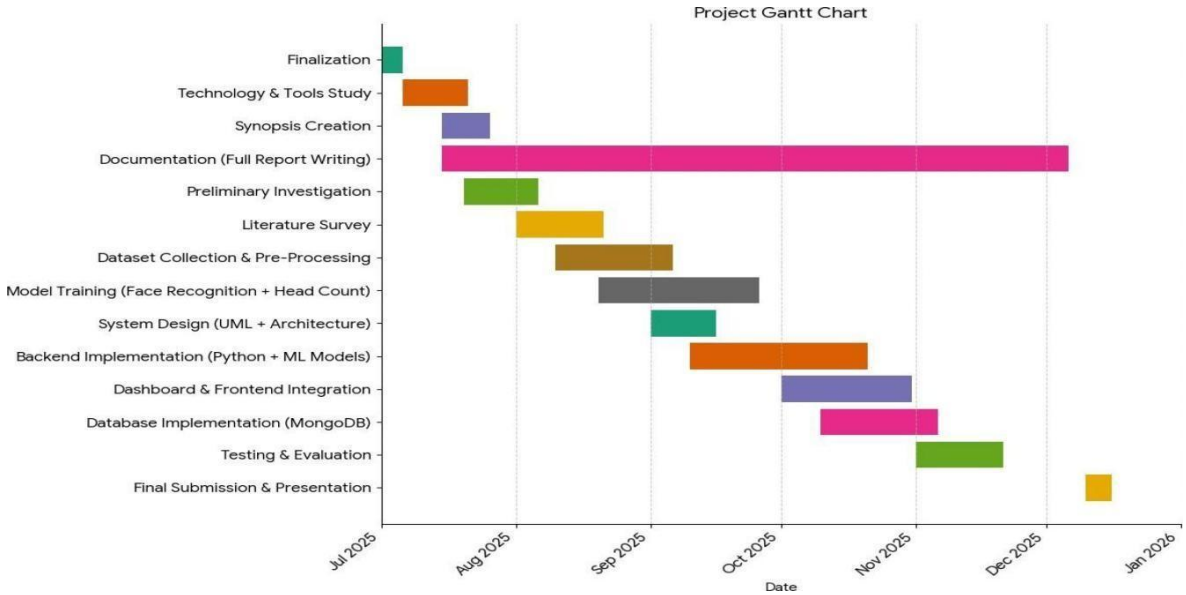
[13] Zhang, K. (2016). Joint Face Detection and Alignment Using CNN. *IEEE Signal Processing Letters*.

[14] Flask Documentation. (2024). Python Microframework for Web Applications.

[15] MongoDB Documentation. (2024). NoSQL Database System

Project Plan

Gantt Chart



Guide Interaction Sheet

Date	Discussion / Task	Action Plan / Outcome
05/07/2025	Final discussion on project domain and selection of idea	*Drishti – AI-Driven Smart Surveillance with Face Recognition & Head Counting* finalized
12/07/2025	Discussion on system requirements and feasibility	Decided to use Python, OpenCV, CNN-based face recognition, and YOLO for detection
20/07/2025	Research on existing systems and literature survey	Multiple research papers reviewed, key gaps identified
30/07/2025	Creation of the project synopsis	Synopsis drafted and finalized with modules explained

05/08/2025	Finalization of system architecture and UML diagrams	Architecture diagram, use case diagram, and sequence flow completed
15/08/2025	Dataset preparation and integration planning	Decided to use live camera feed + custom dataset for training
25/08/2025	Model selection discussion	YOLOv8 for headcounting & LBPH/MobileNet for face recognition chosen
05/09/2025	Backend setup discussion	MongoDB selected for storing logs, attendance, alerts
15/09/2025	Implementation of face recognition module	Face recognition pipeline successfully developed
25/09/2025	Implementation of head counting system	YOLO-based people detection integrated and tested
05/10/2025	Integration discussion	Combined face recognition + headcounting + database logging
15/10/2025	Dashboard design and UI planning	Real-time feed, detection logs, analytics charts added
25/10/2025	System-level testing initiated	Conducted functional, black-box, and load tests
05/11/2025	Discussion on errors and improvements	Improved accuracy, reduced false detections, optimized FPS
15/11/2025	Documentation planning	Chapter drafting for methodology, implementation, UML
25/11/2025	Final review of project demonstration	Real-time face recognition + headcount dashboard fully functional
05/12/2025	Final project documentation completion	Report assembled, checked, and ready for submission
10/12/2025	Final presentation and viva preparation	PPT, demo video, and explanation aligned
15/12/2025	Project completed	Project submitted & demonstrated

Table 5.1 Bibliograph

Source Code

 # src/surveillance_system.py

```
# surveillance_system.py

import cv2
from ultralytics import YOLO
from deepface import DeepFace
import pickle
import numpy as np
from database import increment_detection_count, get_person_by_name

# SETTINGS
YOLO_MODEL = "yolov8n.pt"
DB_FILE = "face_db.pkl"
THRESHOLD = 0.35  # Face similarity threshold


# ------------------- HELPERS -------------------

def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))


def load_face_db():
    with open(DB_FILE, "rb") as f:
        return pickle.load(f)


def recognize_face(face_img, face_db):
    """Return recognized person's name or 'Unknown'"""
    try:
        reps = DeepFace.represent(face_img, model_name="Facenet",
enforce_detection=False)
        if not reps:
            return "Unknown"

        embedding = reps[0]["embedding"]
        best_match, best_score = "Unknown", -1

        for person, db_embeddings in face_db.items():
            for db_emb in db_embeddings:
                score = cosine_similarity(embedding, db_emb)
                if score > best_score:
                    best_score, best_match = score, person
```

```
        return best_match if best_score >= THRESHOLD else "Unknown"

    except:
        return "Unknown"


# ------------------- MAIN -------------------

def main():

    model = YOLO(YOLO_MODEL)
    face_db = load_face_db()

    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    if not cap.isOpened():
        print("Cannot open camera")
        return

    print("Press 'q' to quit")
    frame_count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        results = model(frame, verbose=False)
        boxes = results[0].boxes
        person_count = 0

        for box in boxes:
            cls = int(box.cls[0])

            if cls == 0:  # person
                person_count += 1
                x1, y1, x2, y2 = map(int, box.xyxy[0])

                # Crop face
                face_crop = frame[y1:y2, x1:x2]
                name = recognize_face(face_crop, face_db) if face_crop.size > 0 else
"Unknown"

                # Update DB if recognized
                if name != "Unknown":
                    face_id = get_person_by_name(name)
                    if face_id:
                        increment_detection_count(face_id, metadata=f"Frame {frame_count}")

                # Draw box and label
```

```
            color = (0, 255, 0) if name != "Unknown" else (0, 0, 255)
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.putText(frame, name, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

        # Display person count
        cv2.putText(frame, f"Person Count: {person_count}", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 0, 255), 2)

        cv2.imshow("Smart Surveillance System", frame)
        if cv2.waitKey(10) & 0xFF == ord("q"):
            break

        frame_count += 1

    cap.release()
    cv2.destroyAllWindows()
    print("Surveillance ended")


if __name__ == "__main__":
    main()
```

 # face_detection.py

```
# face_detection.py

from ultralytics import YOLO
import cv2


def main():
    # Load YOLO model
    model = YOLO("yolov8n.pt")

    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    if not cap.isOpened():
        cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Cannot open camera")
        return

    print("Press 'q' to quit")

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Cannot read frame")
            break
```

```
        results = model(frame, verbose=False)
        boxes = results[0].boxes

        person_count = 0

        for box in boxes:
            cls = int(box.cls[0])

            if cls == 0:  # person
                person_count += 1
                x1, y1, x2, y2 = map(int, box.xyxy[0])

                cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                cv2.putText(frame, "Person", (x1, y1 - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        cv2.putText(frame, f"Person Count: {person_count}", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 0, 255), 2)

        cv2.imshow("Person Detection & Count", frame)

        if cv2.waitKey(10) & 0xFF == ord("q"):
            break

    cap.release()
    cv2.destroyAllWindows()
    print("Capture ended")


if __name__ == "__main__":
    main()
```

#**livecapturemain.py**

```
# livecapture_main.py

import cv2
import time
import threading
import queue
import pickle
import numpy as np
from ultralytics import YOLO
from deepface import DeepFace

# CONFIG
YOLO_MODEL = "yolov8n.pt"
DB_FILE = "face_db.pkl"
CAMERA_INDEX = 0
FRAME_QUEUE_MAX = 4
SIMILARITY_THRESHOLD = 0.35
```

```
USE_CV_CAP_DSHOW = True
SHOW_FPS = True

frame_queue = queue.Queue(maxsize=FRAME_QUEUE_MAX)
stop_event = threading.Event()


# ------------------- HELPER FUNCTIONS -------------------

def load_face_db(path):
    """Load face database from pickle file"""
    try:
        with open(path, "rb") as f:
            db = pickle.load(f)

        for k, lst in db.items():
            db[k] = [np.array(e, dtype=np.float32) for e in lst]

        print(f"[INFO] Loaded {len(db)} identities from DB.")
        return db

    except FileNotFoundError:
        print(f"[WARN] Face DB not found at {path}. Starting empty DB.")
        return {}

    except Exception as e:
        print(f"[ERROR] Failed to load DB: {e}")
        return {}


def cosine_similarity(a, b):
    a = np.array(a, dtype=np.float32)
    b = np.array(b, dtype=np.float32)

    if np.linalg.norm(a) == 0 or np.linalg.norm(b) == 0:
        return -1.0

    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))


def recognize_face(face_img, face_db):
    """Returns (name, similarity score)"""
    try:
        reps = DeepFace.represent(face_img, model_name="Facenet",
enforce_detection=False)
        if not reps:
            return "Unknown", -1.0

        embedding = reps[0]["embedding"]
```

```
    except:
        return "Unknown", -1.0

    best_name = "Unknown"
    best_score = -1.0

    for name, emb_list in face_db.items():
        for db_emb in emb_list:
            score = cosine_similarity(embedding, db_emb)
            if score > best_score:
                best_score = score
                best_name = name

    return (best_name, best_score) if best_score >= SIMILARITY_THRESHOLD
else ("Unknown", best_score)


# ------------------- THREAD FUNCTIONS -------------------

def capture_thread_fn(cap):
    while not stop_event.is_set():
        ret, frame = cap.read()
        if not ret:
            time.sleep(0.1)
            continue

        try:
            frame_queue.put(frame, timeout=0.5)
        except queue.Full:
            try:
                frame_queue.get_nowait()
            except queue.Empty:
                pass
            finally:
                frame_queue.put(frame)

    print("[INFO] Capture thread stopped.")


def worker_thread_fn(model, face_db):
    last_time = time.time()
    fps = 0

    while not stop_event.is_set():

        try:
            frame = frame_queue.get(timeout=0.5)
        except queue.Empty:
            continue
```

```
        h, w = frame.shape[:2]
        results = model(frame, verbose=False)
        boxes = results[0].boxes

        person_count = 0

        for box in boxes:

            cls = int(box.cls[0])
            if cls != 0:
                continue

            person_count += 1
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())

            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

            # Crop top 60% of bounding box as face
            face_y2 = y1 + int((y2 - y1) * 0.6)
            face_crop = frame[y1:face_y2, x1:x2]

            name, score = recognize_face(face_crop, face_db)
            display = f"{name} ({score:.2f})" if score >= 0 else "Unknown"

            cv2.putText(frame, display, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 255), 2)

        # Person count
        cv2.putText(frame, f"Person Count: {person_count}",
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 0, 255), 2)

        # FPS display
        if SHOW_FPS:
            now = time.time()
            dt = now - last_time
            fps = (1 / dt)
            last_time = now

            cv2.putText(frame, f"FPS: {fps:.1f}", (10, 60),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

        cv2.imshow("Drishti - Live Feed", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            stop_event.set()
            break

    print("[INFO] Worker thread stopped.")


# ------------------- MAIN -------------------
```

```
def main():

    print("[INFO] Loading YOLO model...")
    model = YOLO(YOLO_MODEL)

    print("[INFO] Loading face DB...")
    face_db = load_face_db(DB_FILE)

    # Open camera
    if USE_CV_CAP_DSHOW:
        cap = cv2.VideoCapture(CAMERA_INDEX, cv2.CAP_DSHOW)
        if not cap.isOpened():
            cap = cv2.VideoCapture(CAMERA_INDEX)
    else:
        cap = cv2.VideoCapture(CAMERA_INDEX)

    if not cap.isOpened():
        print("[ERROR] Could not open camera.")
        return

    t1 = threading.Thread(target=capture_thread_fn, args=(cap,), daemon=True)
    t2 = threading.Thread(target=worker_thread_fn, args=(model, face_db),
daemon=True)

    t1.start()
    t2.start()

    try:
        while not stop_event.is_set():
            time.sleep(0.2)
    except KeyboardInterrupt:
        stop_event.set()

    t1.join()
    t2.join()

    cap.release()
    cv2.destroyAllWindows()

    print("[INFO] Application terminated.")


if __name__ == "__main__":
    main()
```